Approximate Analysis by Synthesis

Lecture 19
Notes for Probabilistic Models of the Visual Cortex

Alan Yuille

Johns Hopkins University

5 November, 2026

What this lecture covers

Analysis by synthesis

Robust vision and 3D models

Neural mesh models

Inference and robustness

Summary

Colors used in the figures

These notes follow Yuille's 2025 slides of this lecture, which these notes call A (Yuille, A). A caption citing "A p. *N*" took its picture from page *N* of A. Many of these pictures come from the papers discussed here, and the captions name them.

In the diagrams drawn for these notes and the slides, colors only tell elements apart, and each caption says what they mark. The same holds for the figures reproduced from the papers: for example, in the occlusion maps of the neural mesh models green marks the visible object and red the part predicted occluded, while in the score maps green is the object model's score and orange the background model's.

Analysis by synthesis inverts the synthesis of images

An image I is produced by a state of the world W: the objects in the scene, their 3D (three-dimensional) shapes, poses and materials, and the lighting. The process that produces the image from the world is called *synthesis*, and a probabilistic model of it is the likelihood $P(I \mid W)$: how probable the image I is if the world is in state W. Knowledge of which world states are common is the prior $P(W)$.

Vision goes the other way: from the image to the world. Analysis by synthesis performs this analysis by inverting the synthesis with Bayes' rule,

$$P(W \mid I) = \frac{P(I \mid W)\, P(W)}{P(I)},$$

and reports the world states with high posterior probability $P(W \mid I)$. The idea was first formulated by Ulf Grenander in the 1960s in terms of Bayesian probability theory (Grenander, 1978), and developed as pattern theory (Mumford, 1994). Yuille and Kersten (2006) review it as a theory of human vision.

Analysis by synthesis inverts the synthesis of images

The approach differs from direct prediction of $P(W \mid I)$ by standard discriminators, such as a neural network trained to map images to labels. A discriminator learns the posterior from examples; analysis by synthesis builds it from a model of how images are made.

The example in the figure is due to Sinha and Adelson (1993). Picture (a) is a line drawing in the image plane: a hexagon with three inner lines. Many 3D configurations project onto exactly this drawing; picture (c) stacks several of them above the image plane, and the dotted vertical lines show that each of their corners projects straight down onto a corner of the drawing. All of them explain the image equally well, so the image alone cannot decide between them. People see one of them, the cube of picture (b), which is the configuration the posterior prefers once regular shapes, such as boxes with right angles, are more probable a priori. The figure keeps the labels of A p. 3: "likelihoods $P(I \mid W)$" over shaded 3D shapes, candidate objects whose images the likelihood compares with the drawing, and "prior $P(W)$" over the stack (c) of configurations, among which the prior chooses.

Analysis by synthesis inverts the synthesis of images

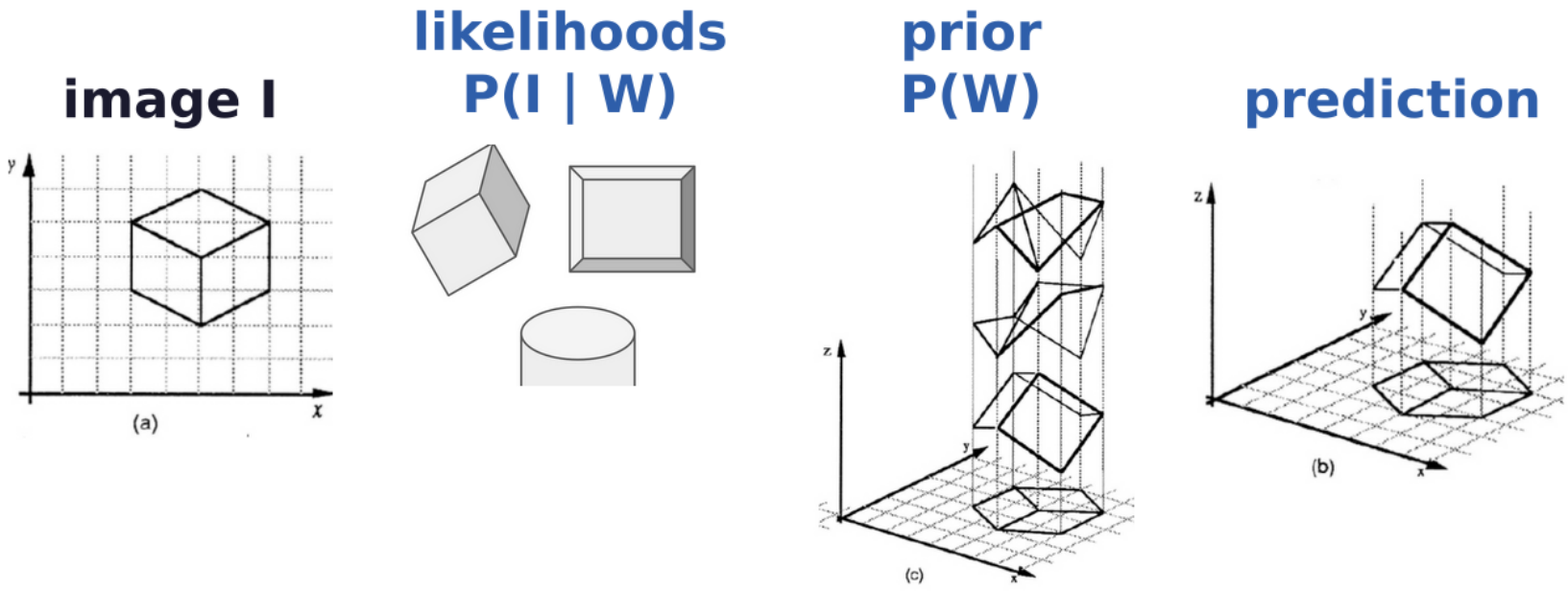


Figure 1: The four pictures of A p. 3 with its labels: (a) the image, a line drawing; three shaded 3D shapes; (c) several 3D configurations that project onto the drawing (dotted lines: the projection); (b) the interpretation people see, a cube. After Sinha and Adelson (1993).

Low-level vision is ambiguous; high-level vision rarely is

Low-level vision extracts local properties of the image, such as edges: places where the intensity changes sharply. Seen through a small window, such a property is ambiguous. An edge on a face and an edge on a tree can look the same, so it is difficult to determine whether an edge belongs to a face or to a tree from the edge alone.

High-level vision interprets the image as objects, and this is rarely ambiguous. The patterns of objects are complex: a face has eyes, a nose and a mouth in a fixed arrangement, and such a pattern rarely occurs by chance. In the photograph of the figure, nobody mistakes the face for the tree, although small patches cut from the two can hardly be told apart.

Low-level vision is ambiguous; high-level vision rarely is

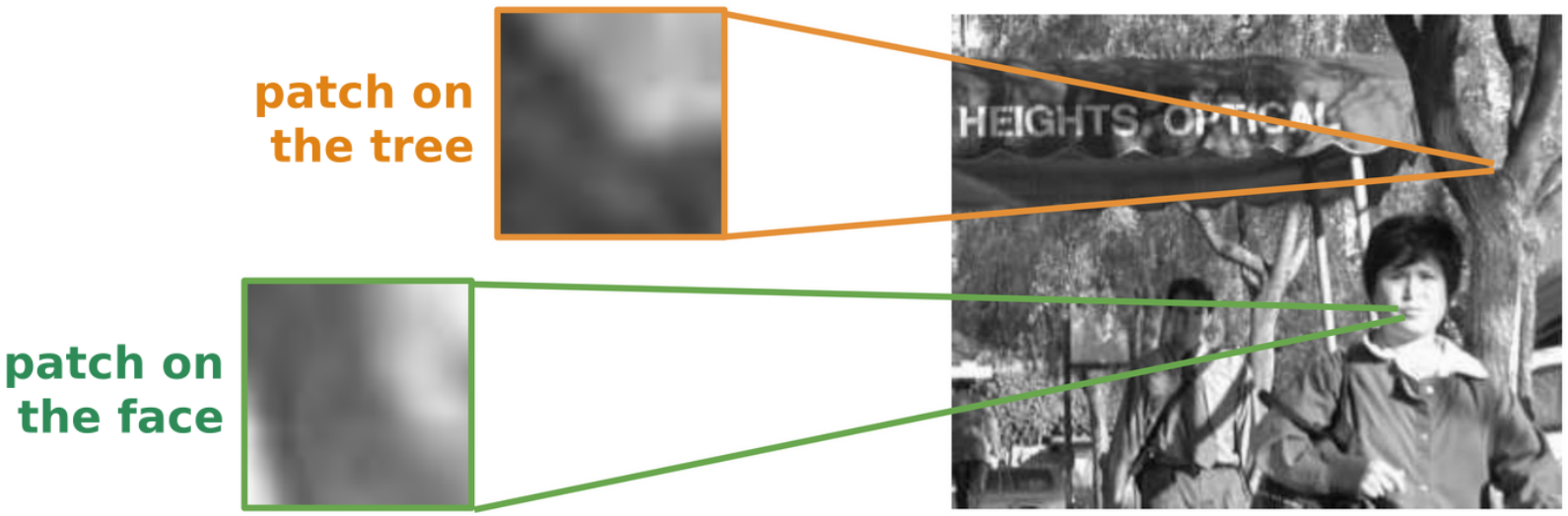


Figure 2: A photograph of a person by a tree (A p. 4). Two small patches are enlarged: one cut from the tree (orange box and lines) and one from the face (green). The enlargements are blurred because they magnify only a few pixels.

Low-level cues propose; high-level models validate

The question is how to use low-level cues to access the correct high-level models, and so resolve the low-level ambiguities. Searching through every object model for every image is too slow. Analysis by synthesis combines two passes. Bottom-up, from the image upward, the low-level cues make proposals: they point to the few high-level models that could have produced them. Top-down, from the models back to the image, the high-level models validate the proposals.

The example is a letter. The image is a black F. Feature extraction finds its pieces: two short horizontal bars (red), a long vertical bar and a small mark (blue). These features propose letters and other patterns that contain them: the vertical bar with two horizontal bars fits an E or an F, and the two horizontal bars alone fit parallel bars (an equals sign). A column of candidate patterns (letters, bars, marks) stands beside the image; the circled ones are proposed, and the dots stand for the many candidates left out.

Low-level cues propose; high-level models validate

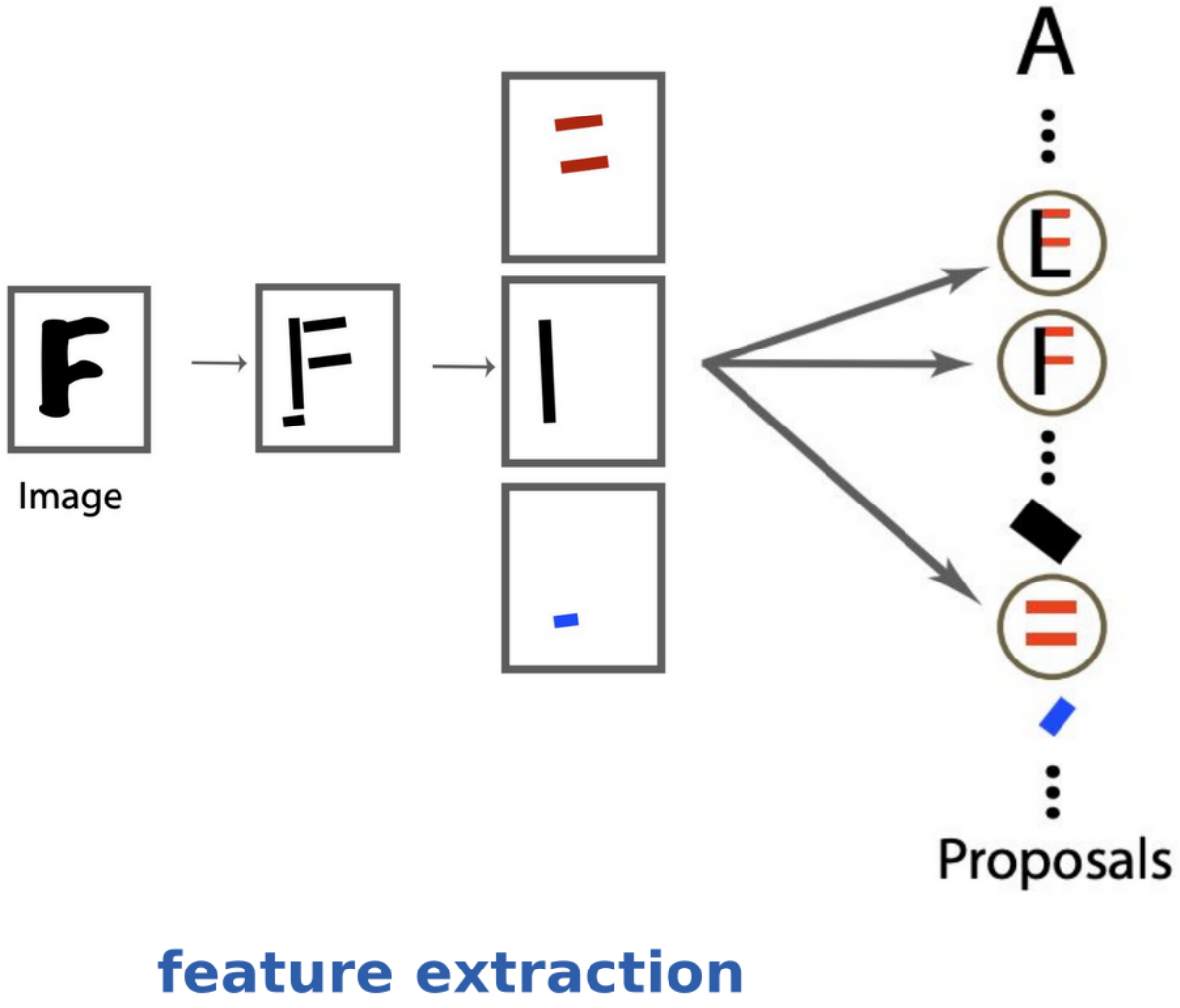


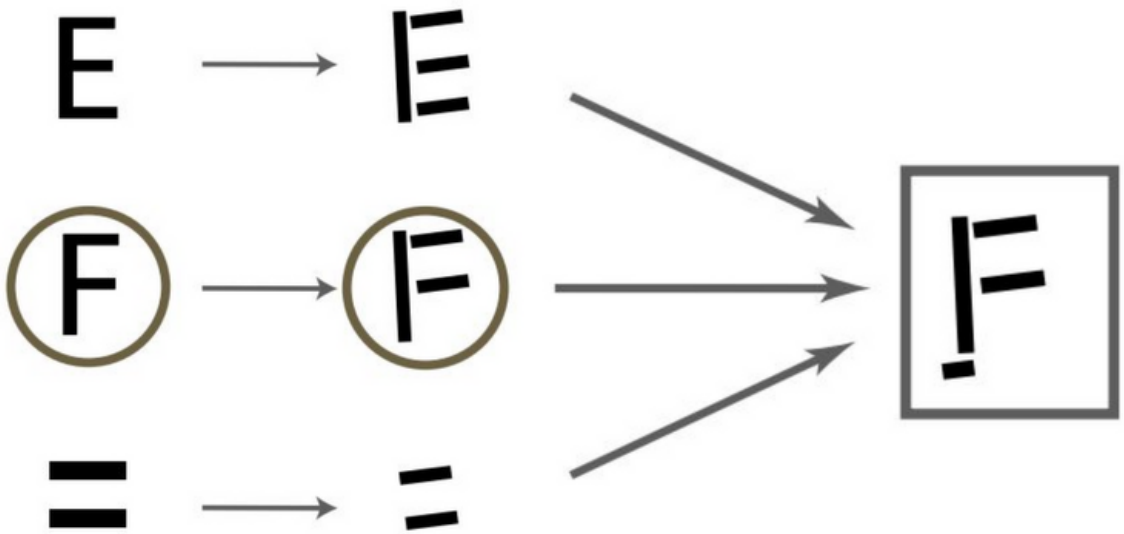
Figure 3: Feature extraction and proposals (A p. 6, after Yuille and Kersten, 2006, Figure 2). From the image (the F), the features are extracted (the three boxes: two horizontal bars, a vertical bar, a small mark); the arrows lead to the proposals (circled: E, F, an equals sign).

Synthesis and verification: the F explains the features

Each proposal is then checked top-down. A proposed letter synthesizes, from its model, the features it would produce in the image, and these are compared with the features that were extracted.

An E predicts a third horizontal bar at the bottom, which the image lacks. The parallel bars predict the two horizontal bars and leaves the vertical bar unexplained. The F predicts a vertical bar with two horizontal bars at its top, which is what the image contains: it explains almost all the features, so it is preferred to the E and to the parallel bars (Yuille and Kersten, 2006). The box at the end of the figure is the resulting interpretation of the image: an F.

Synthesis and verification: the F explains the features



synthesis and verification

Figure 4: Synthesis and verification (A p. 7). Each row starts from a proposal (E, F, parallel bars) and shows the features it synthesizes; the circled F is the proposal that is kept, and the arrows lead to the interpretation of the image.

Analysis by synthesis is harder when the world is complex

How hard analysis by synthesis is depends on how complex the world is. Yuille and Kersten (2006) illustrate this with three worlds that generate images of letters.

In world A, a simple vocabulary generates the image: the vocabulary has only whole letters, so there is little or no ambiguity in interpreting the image. In world B the vocabulary is richer: letters, bars and fragments, so a given cause, such as a particular letter, can appear in many different images, and different causes can produce similar ones. In world C the letters are also distorted, overlapped by bars and fragments, and drawn on a slanted surface. The richer the vocabulary, the greater the image ambiguity, and the harder it is to interpret the image. Analysis by synthesis is easy in world A, harder in world B and much harder in world C, and natural images are closer to world C.

Analysis by synthesis is harder when the world is complex

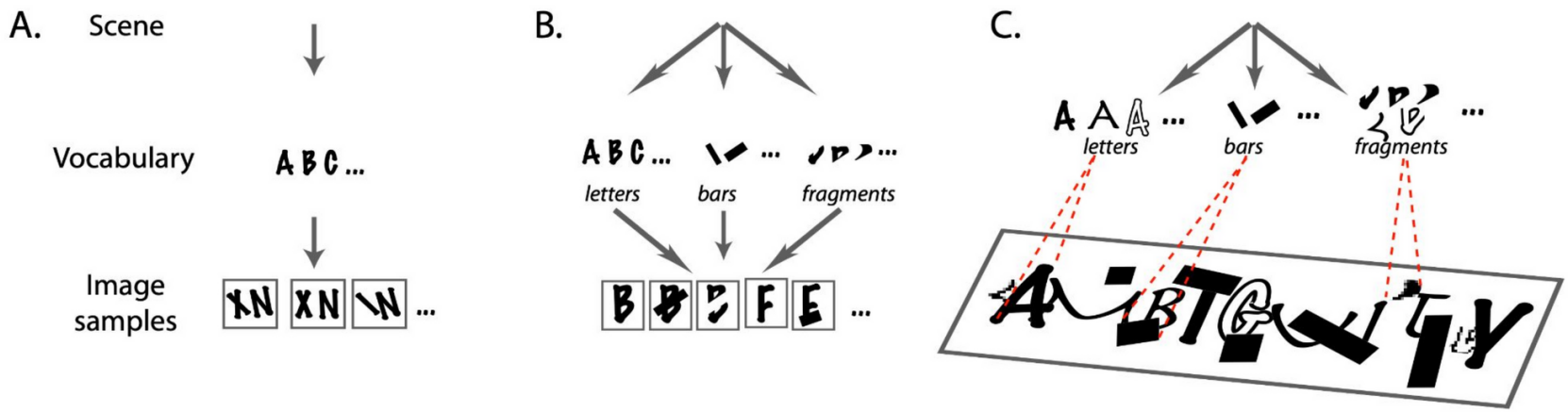


Figure 5: Three worlds (A p. 8; Yuille and Kersten, 2006, Figure 1). A: a scene of letters, a vocabulary, and the image samples it generates. B: a vocabulary of letters, bars and fragments. C: the same vocabulary, with distortion and occlusion, on a slanted page; the dotted lines link parts of the image to their causes.

Analysis by synthesis faces three challenges

Analysis by synthesis is conceptually attractive, and three challenges stand in its way.

- *Synthesis.* It requires generative models of the image, conditioned on the world state: models that can render any object, pose and lighting.
- *Analysis.* Inverting the generative model is not easy. Searching over all possible ways an image could be generated is impractical, because the world states are too many and the rendering is a complicated function of them.
- *Evaluation.* Current annotated benchmarks are not designed for evaluating analysis by synthesis. They usually test one task, such as a class label for each image, while analysis by synthesis estimates the whole world state.

Neural mesh models are one answer to the first two challenges: they replace pixels by learned features, so that both the synthesis and its inversion become simple enough to use.

Analysis by synthesis faces three challenges

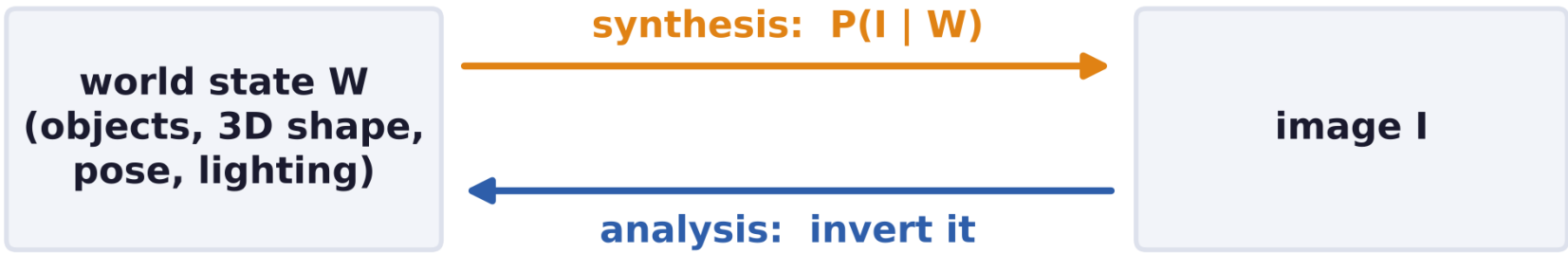


Figure 6: The two directions: synthesis (orange) generates the image from the world state W with probability $P(I \mid W)$; analysis (blue) inverts it.

Robust vision generalizes beyond the training data

The promise of analysis by synthesis is measured against what current vision systems lack, and the first thing is robustness. Human vision is robust: it works in viewing conditions it has never encountered. We recognize a camel covered in snow, although camels are usually seen in deserts; a yellow toy duck seen only through three small holes, which show part of an eye and of the beak; and a spotted animal in a high-contrast black-and-white picture that removes most of its usual appearance. A vision system that only works on images like its training images is not robust in this sense.

An important side note: once we recognize an object, we also know its pose, its parts and its shape. Recognition is part of an interpretation of the whole object. This is the multi-tasking that analysis by synthesis promises, because a model that generates the object's image contains all of these properties at once.

Robust vision generalizes beyond the training data

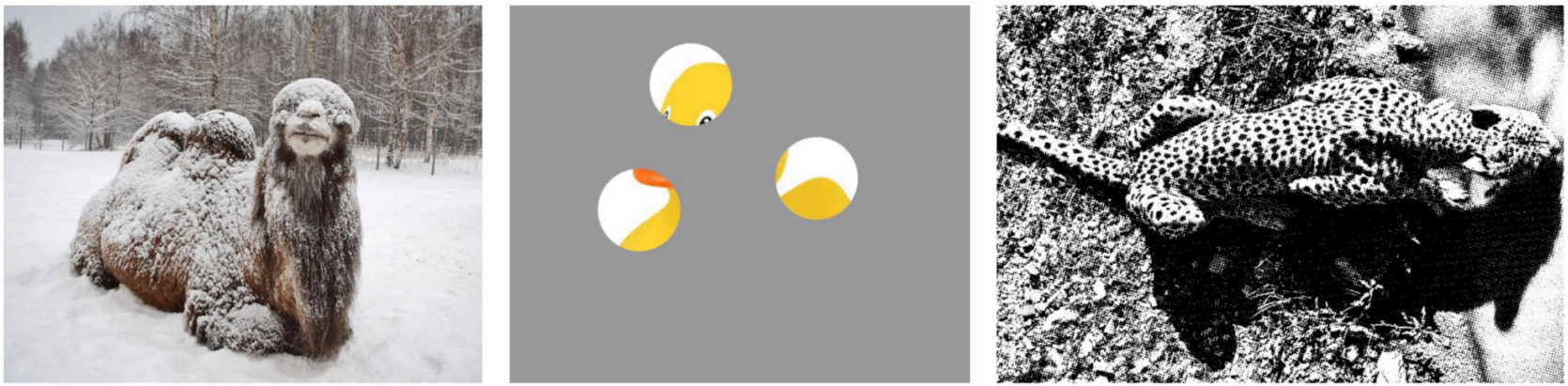


Figure 7: Objects in unusual viewing conditions (A p. 10): a camel in snow, a toy duck seen through three holes in a grey screen, and a spotted animal in a high-contrast picture.

Neural network discriminative models lack robustness

Deep networks trained discriminatively, to map an image to a label, succeed at large-scale visual recognition, yet lack robustness in two ways.

The first is robustness to 3D changes, unusual poses and viewpoints. Qiu and Yuille (2016) rendered a sofa from many viewpoints with UnrealCV, a plugin that connects vision programs to the Unreal Engine game engine; the average precision (the standard accuracy measure of detection) of Faster R-CNN (region-based convolutional neural network) ranged from 0.1 to 1.0 with the viewpoint. Alcorn et al. (2019) fooled image classifiers by rendering familiar objects, such as a school bus, in strange poses.

The second is robustness to changes of image components. Placing an object in an unexpected context changes the detections of the other objects (Rosenfeld et al., 2018); networks trained on ImageNet, a large dataset of labeled photographs, rely on texture more than on shape (Geirhos et al., 2019); detectors degrade in snow, frost and fog (Michaelis et al., 2019); and they fail when objects are partly occluded (Kortylewski et al., 2020).

Neural network discriminative models lack robustness

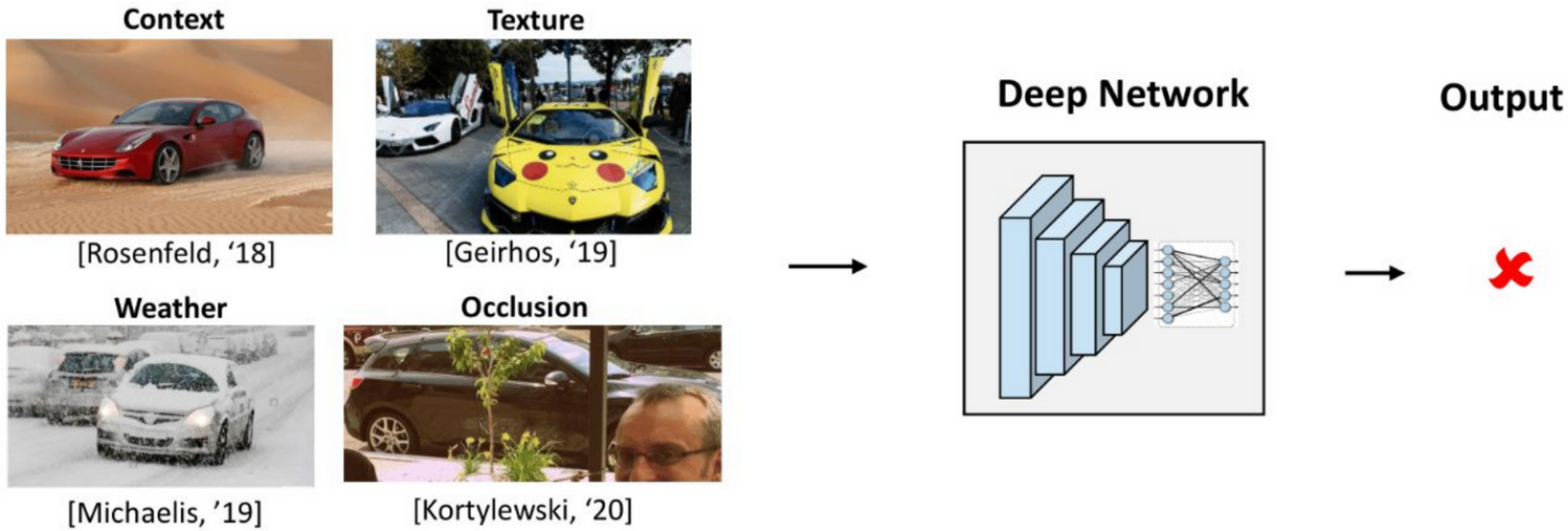


Figure 8: Four kinds of change that make a deep network fail (A p. 11): context (a sports car in a desert), texture (a car painted with a cartoon pattern), weather (a car in snow) and occlusion (a car behind a plant and a person). The red cross marks a wrong output.

Neural network generative models concentrate on synthesis

Recent neural network generative models, such as GANs (generative adversarial networks) and diffusion models, can generate amazing images. Are they strong generators for analysis by synthesis?

These models concentrate on synthesis, generating images, instead of analysis. Most of them are conditioned on latent variables (random vectors z that index the images the model can produce) or on text embeddings (vectors computed from a sentence that describes the image). Analysis by synthesis requires generative models conditioned on the state W of the 3D world: the shapes, poses, textures and lighting of the objects. Only then does inverting the model recover W, which is what vision needs. Inverting a model conditioned on z recovers z, which has no fixed meaning in the world.

Neural network generative models concentrate on synthesis

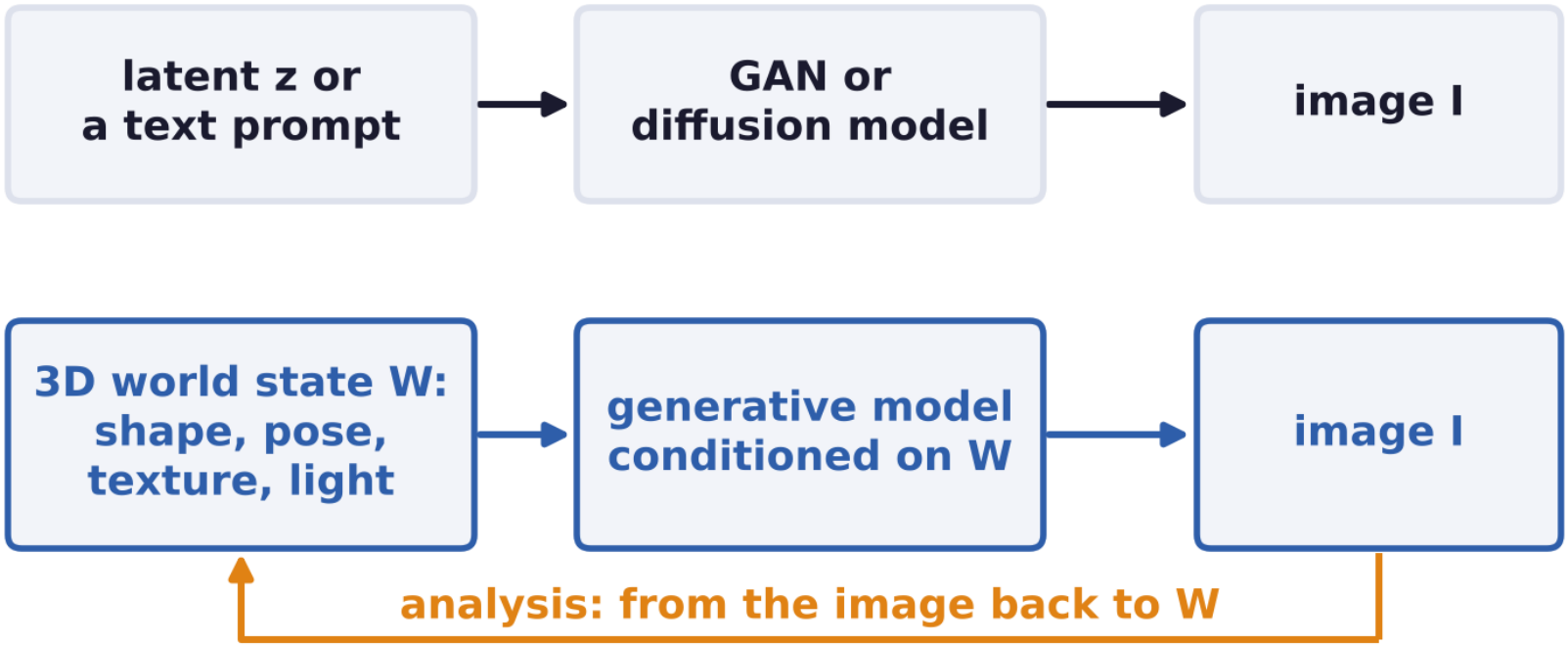


Figure 9: Two kinds of conditioning. The upper row: today's image generators map latent variables or a prompt to an image. The lower row (blue): analysis by synthesis needs a model conditioned on the 3D world state W, so that analysis (orange) can go from the image back to W.

Computer vision via analysis by synthesis inverts graphics

Computer graphics synthesizes a 2D image from a 3D scene: given the geometry, the materials, the lights and the camera, it computes the image the camera would record. This is the forward model $P(I \mid W)$ of analysis by synthesis, made precise.

Computer vision via analysis by synthesis inverts it: it finds the 3D scene that synthesizes the image. Vision systems of this kind analyze images by synthesizing them, a program that goes back to Grenander (1978), Mumford (1994) and Yuille and Kersten (2006).

Computer vision via analysis by synthesis inverts graphics

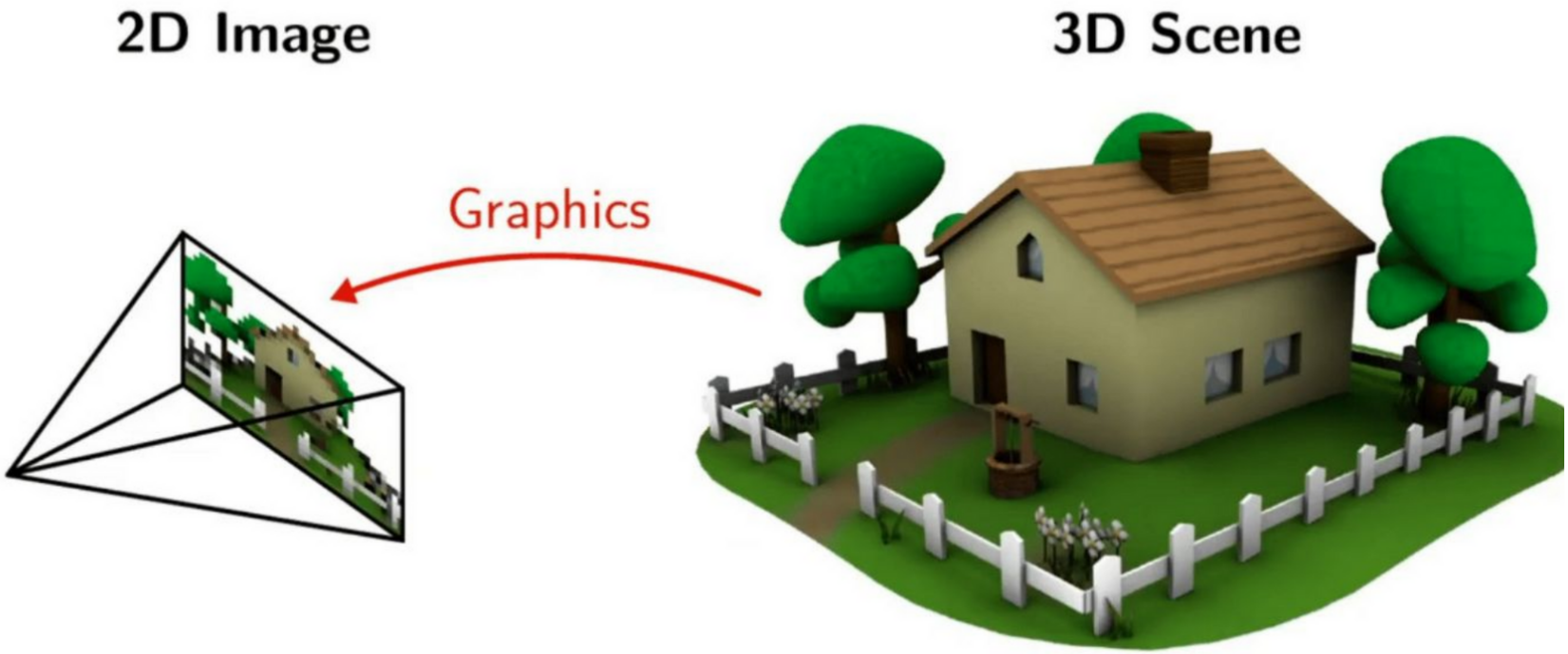


Figure 10: Graphics (red arrow) projects a 3D scene, a house with trees and a fence, onto the image plane of a camera (the pyramid on the side of the 2D image). A p. 13.

A computer graphics model generates images of the object

For one object, a computer graphics model has a 3D shape (a mesh of vertices and faces), a texture (the colors painted on the surface), and further parameters such as the illumination. Rendering computes its image for a given camera: in ray tracing, a ray is sent from the camera through each pixel of the image plane, and the color of the pixel is that of the surface point the ray hits, lit by the light sources.

Rendering the same model with one parameter changed gives new images of the object: other textures, other 3D poses, other illuminations. Such a model is a generative model of the object's images.

A computer graphics model generates images of the object

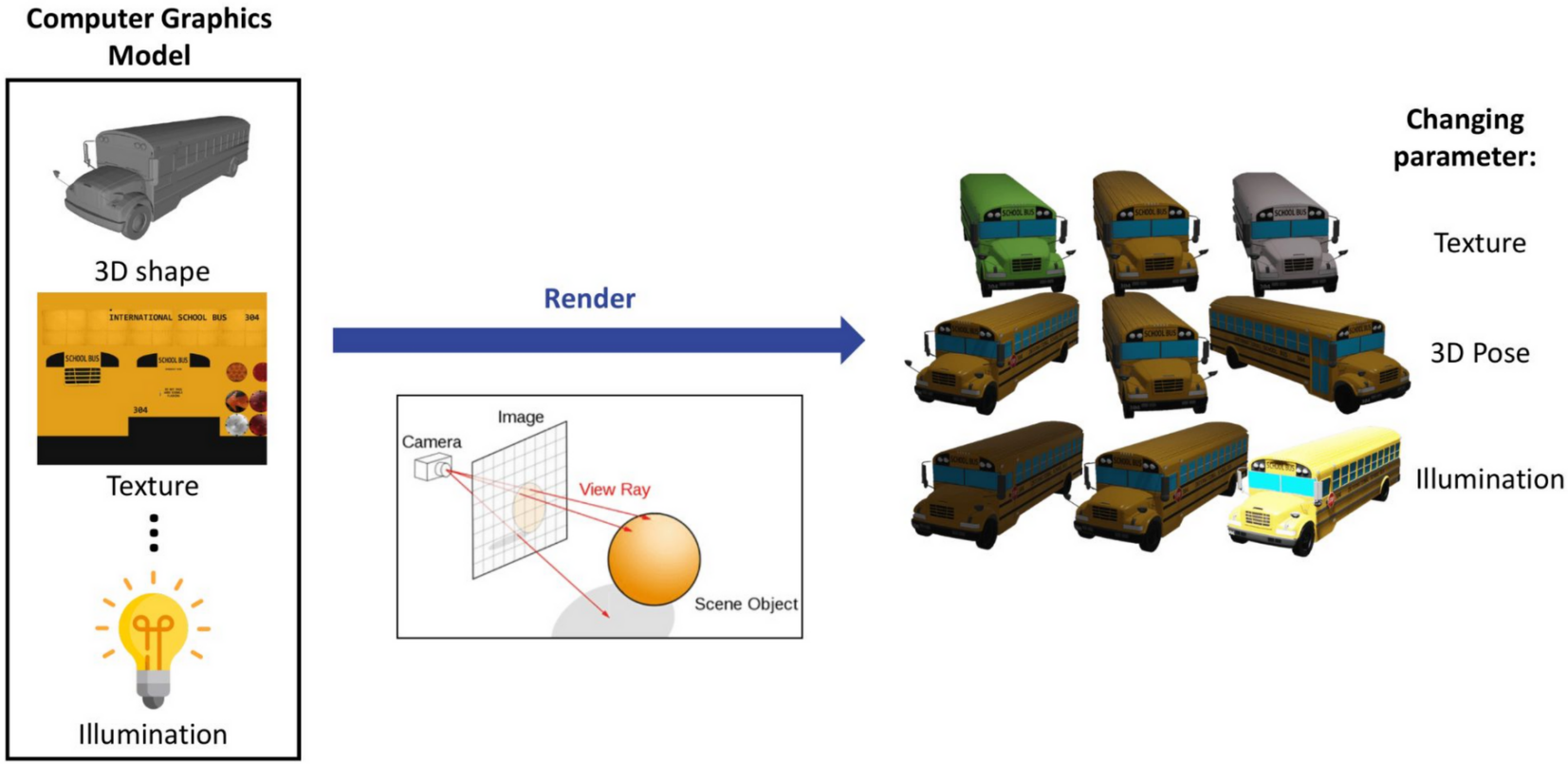


Figure 11: A computer graphics model of a school bus (its 3D shape, its texture, and the illumination) and renderings with a changed texture, 3D pose and illumination. The sketch under the arrow shows ray tracing: a ray from the camera through the image hits the scene object. A p. 14.

Inverse rendering: render, compare, update the parameters

Inverse rendering applies analysis by synthesis to a test image. Start from a guess of the model's parameters, here the 3D pose of the bus. Render the 3D object model with these parameters and compare the rendering with the test image; the difference is measured by a reconstruction loss, for example the sum of squared differences of the pixels. Then update the model parameters to reduce the loss, by a gradient step if the renderer is differentiable or by a search otherwise, and repeat until the rendering matches the image.

The pose found at the end is the estimate. Under the probabilistic view, the reconstruction loss plays the role of $-\log P(I \mid W)$, and the loop searches for the world state that best explains the image (the maximum likelihood); adding $-\log P(W)$ to the loss would give the most probable one.

Inverse rendering: render, compare, update the parameters

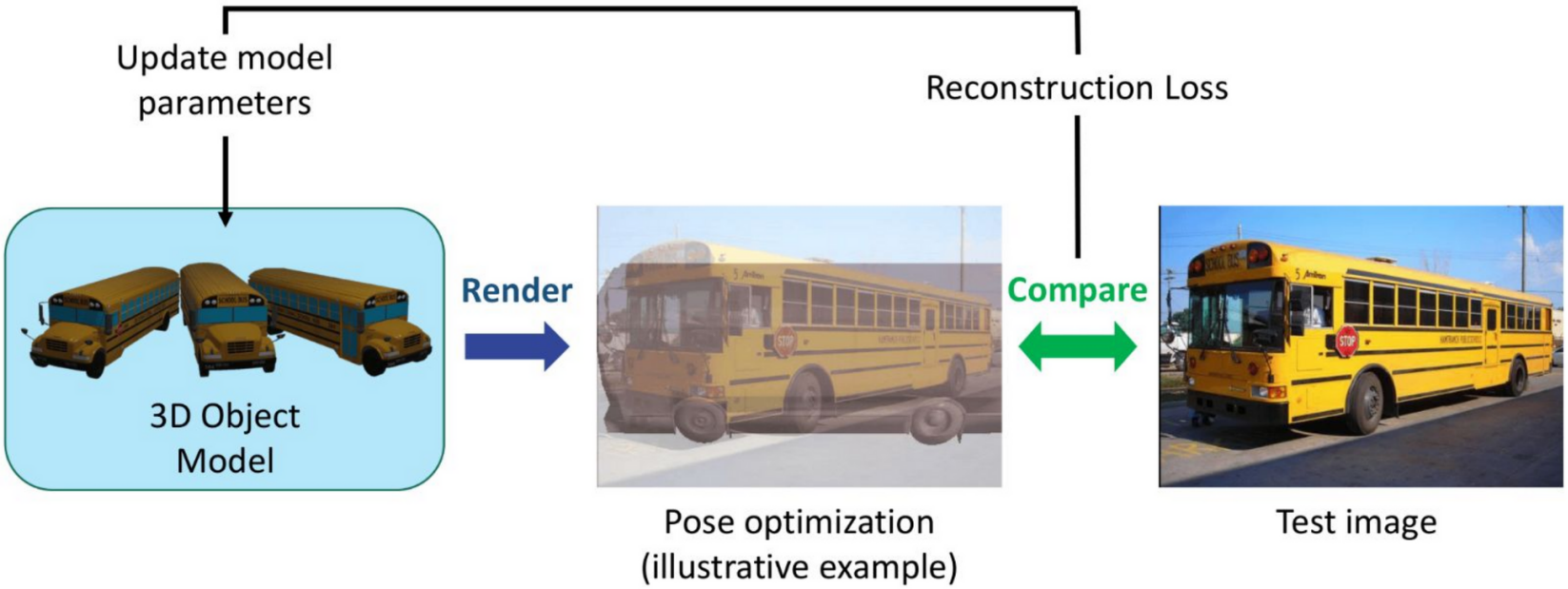


Figure 12: Inverse rendering (A p. 16). The 3D object model is rendered, the rendering (overlaid, semi-transparent, on the image) is compared with the test image, and the reconstruction loss updates the model parameters. The pose is an illustrative example.

Analysis by synthesis is 3D-aware, compositional and robust

Compared with deep networks, analysis by synthesis has three advantages.

- It is 3D-aware and compositional: the model represents the object in 3D, built from parts, so a new pose or a new combination of parts needs few or no new training examples.
- It is robust to occlusion and unseen poses. 3D morphable models of faces (Paysan et al., 2009), statistical models of the 3D shape and texture of faces, are fitted to faces across poses and illuminations, and occlusion-aware versions explain occluders as outliers (Egger et al., 2018); neural mesh models do the same for objects (Wang et al., 2021).
- It is multi-tasking: once the model is fitted, its pose, parts and shape are all known, and one model of the object serves several tasks consistently.

The figure returns to the bus of inverse rendering after the update of the pose: the rendered bus now lies on the bus of the test image, and the fitted model gives the pose and the shape of the bus at once.

Analysis by synthesis is 3D-aware, compositional and robust

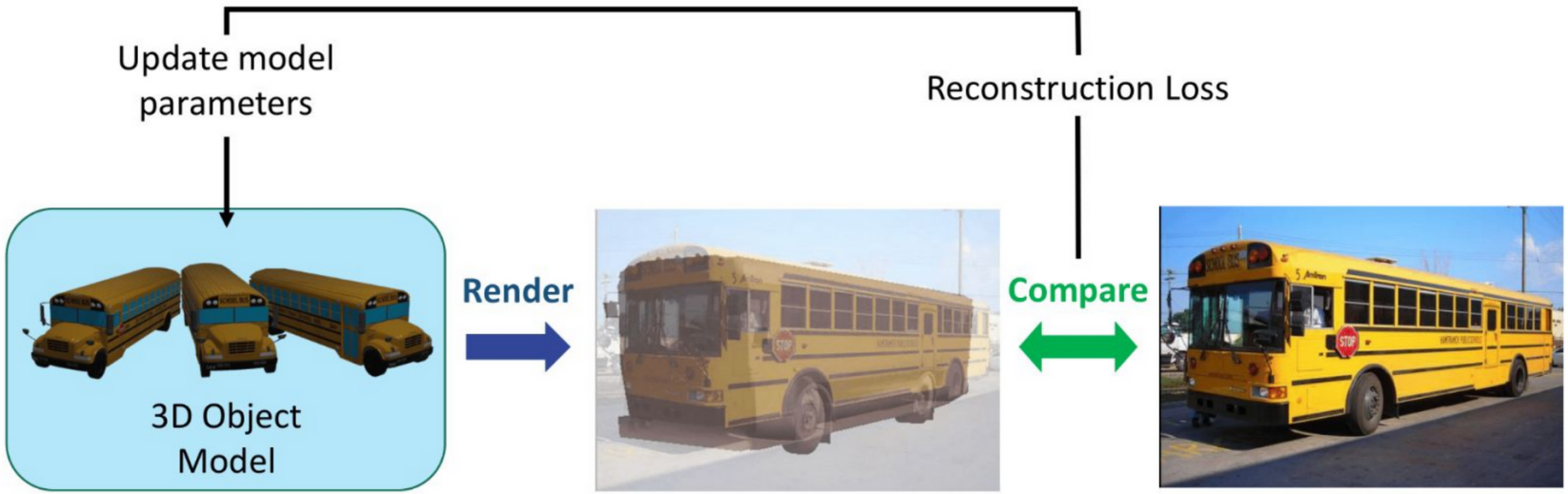


Figure 13: The inverse rendering loop after the update of the pose: the rendered bus now lies on the bus of the test image. A p. 17.

Generative object models are hard to learn and to invert

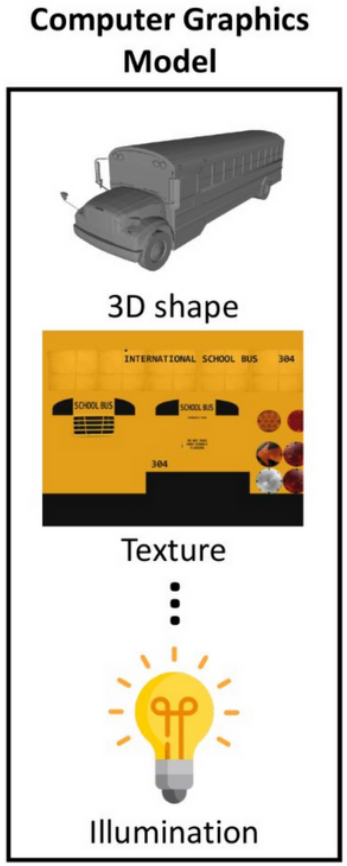
Why is analysis by synthesis not widely used in computer vision? There are two reasons.

First, it is hard to learn the generative object model. A graphics model of a bus, with its exact shape, texture and reflectance, has to be built by hand or learned from many images, and every object category needs its own. Real images also contain clutter and lighting that such a model does not describe.

Second, it is hard to optimize the inverse rendering process. The reconstruction loss over pixels has many local minima: a rendering with a wrong pose may overlap the object in the image only in part, so the gradient gives little guidance, and small differences of texture or lighting dominate the loss.

Generative object models are hard to learn and to invert

1) learn the object model



2) optimize the inverse rendering



Figure 14: The two difficulties (A p. 18): (1) learning the computer graphics model of the object (from A p. 14) and (2) optimizing the inverse rendering (the pose optimization of A p. 16).

Neural analysis by synthesis estimates 3D pose from features

Neural analysis by synthesis keeps the loop of inverse rendering and changes what is rendered and compared: features instead of pixels (Wang et al., 2021). A feature is a vector computed by a neural network at each position of the image; it describes the local appearance in a way that the network has learned.

The pipeline has four parts. A feature extractor, a convolutional neural network, maps the input image to a grid of extracted features. A generative object model, a mesh whose vertices carry feature vectors (the neural texture), is rendered at a camera pose m: each image position covered by the mesh receives the feature of the vertex that projects there. The two feature grids are compared, and the pose m is changed until they match. Both the feature extractor and the neural texture need to be learned; the output is the estimated 3D pose, shown by overlaying the rendered mesh on the input image.

Neural analysis by synthesis estimates 3D pose from features

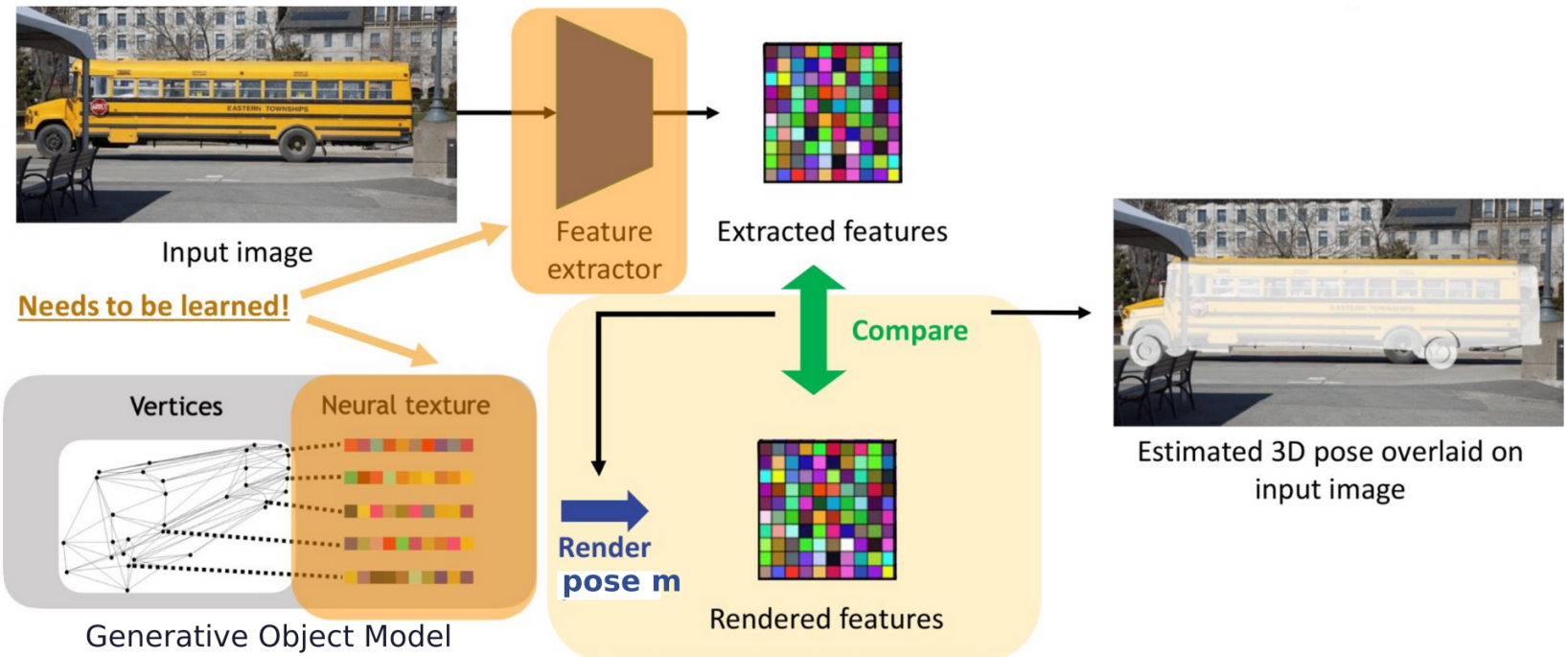


Figure 15: Neural analysis by synthesis (A p. 19). The input image goes through the feature extractor (brown) to the extracted features; the generative object model (vertices and neural texture) is rendered at pose m to the rendered features; the two are compared. "Needs to be learned" marks the extractor and the neural texture.

A neural mesh model stores a feature vector at each vertex

A neural mesh model $\mathfrak{N}_y$ represents an object category y, such as "car", by a mesh of R vertices in 3D. Instead of a color, vertex r stores a feature vector θ_r; together these vectors form a neural texture. The neural mesh model of this lecture is NeMo (Wang et al., 2021); in its official code the features have 128 dimensions.

A CNN (convolutional neural network) maps the image to a feature map F: a grid of feature vectors f_i, one per position i. In NeMo the CNN is a ResNet-50 (a 50-layer residual network) with two extra upsampling layers, and the grid has one position for every 8×8 pixels. Rendering the mesh at a pose m predicts a feature at each position the mesh covers, and these predictions are compared with F instead of the pixels. The comparison is therefore made in a space where the network has already removed much of the variation of color and lighting.

A neural mesh model stores a feature vector at each vertex

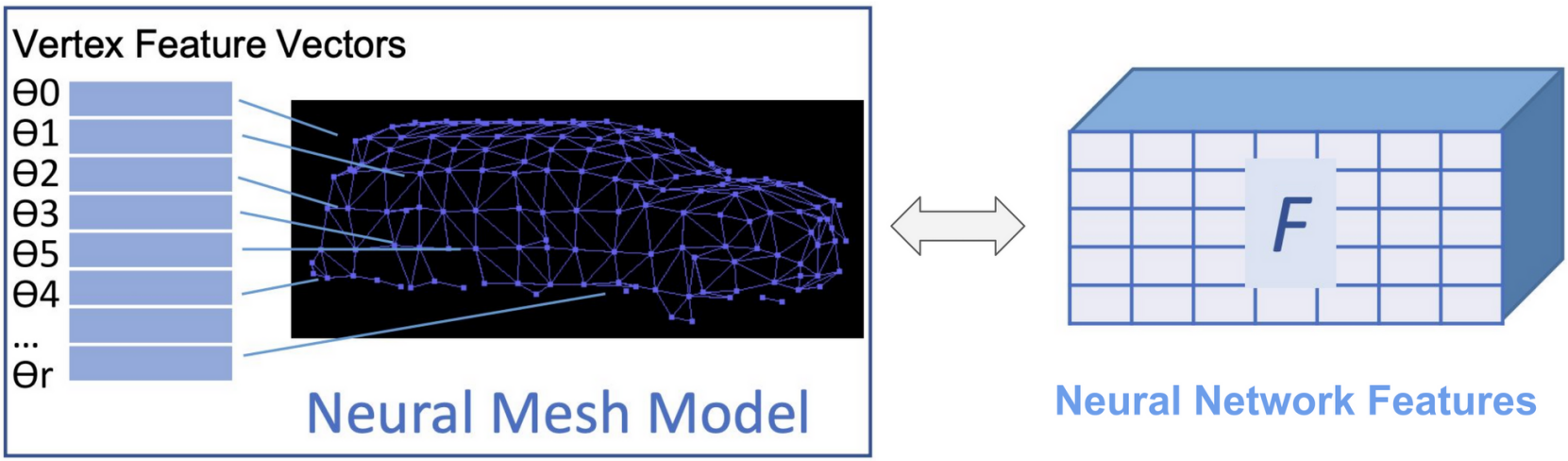


Figure 16: A neural mesh model of a car (A p. 20): a mesh (purple vertices and edges) whose vertices store the vertex feature vectors $\theta_0, \theta_1, \ldots, \theta_r$ (the blue bars; the labels are those of A). It is compared with the neural network features F of the image, drawn as a block of feature vectors.

The likelihood factorizes over foreground and background

The neural mesh model is a generative model of the feature map. Given the mesh $\mathfrak{N}_y$ of category y, the camera pose m and a background model $B = \{\beta, \sigma\}$, the probability of the feature map is (Wang et al., 2021)

$$p(F \mid \mathfrak{N}_y, m, B) = \prod_{i \in \mathcal{FG}} p(f_i \mid \mathfrak{N}_y, m) \prod_{i' \in \mathcal{BG}} p(f_{i'} \mid B).$$

The foreground $\mathcal{FG}$ is the set of positions of the feature map covered by the mesh rendered at pose m; the background $\mathcal{BG}$ is the set of all the other positions. Each position contributes one factor: a foreground position i is explained by the mesh, a background position i' by the background model. The product means that, given the mesh and the pose, the features are treated as independent.

The likelihood factorizes over foreground and background

Both factors are Gaussians:

$$p(f_i \mid \mathfrak{N}_y, m) = \frac{1}{\sigma_r \sqrt{2\pi}} \exp\Big(- \frac{\|f_i - \theta_r\|^2}{2\sigma_r^2} \Big),$$

$$p(f_{i'} \mid B) = \frac{1}{\sigma \sqrt{2\pi}} \exp\Big(- \frac{\|f_{i'} - \beta\|^2}{2\sigma^2} \Big).$$

Here r is the vertex that projects to position i, θ_r its feature and σ_r the spread of the image features around it. The background model has one clutter feature β, a single vector for everything that is not the object, with spread σ. (The constants are written as in the paper; for D-dimensional features they are $(\sigma\sqrt{2\pi})^{-D}$, and with all spreads equal to 1, as in training, they are the same for every factor and drop out.)

The negative logarithm of the likelihood is a sum of squared distances. With all spreads equal to 1, maximizing the likelihood means making each foreground feature close to its vertex feature and each background feature close to β: this is the loss $\mathcal{L}_{ML}$ of the training.

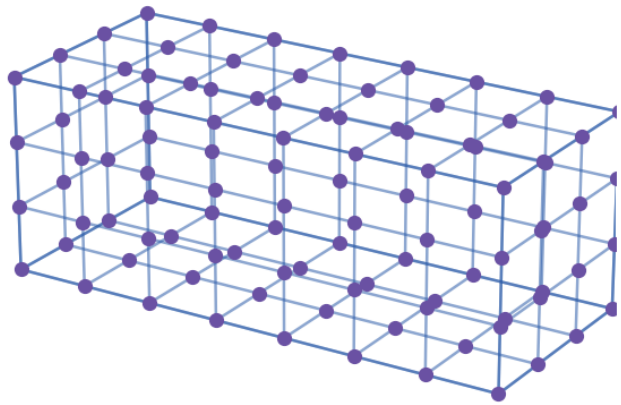
The approximations: features, factorization, weak geometry

The model is an approximate analysis by synthesis, in three ways.

- It generates features instead of pixels. The renderer is simple, and the hard part of appearance (texture, lighting, materials) is left to the feature extractor.
- Factorization: the features are treated as independent of each other, given the mesh and the pose, although neighboring features of a real image are strongly correlated.
- Weak geometry: a cuboid is used for aligning instances of the same category. The mesh does not follow the exact shape of each car; a box with vertices on a regular grid over its faces only needs to align corresponding parts of different cars. NeMo-SingleCuboid uses one cuboid per category and NeMo-MultiCuboid one per subtype (a subtype is one of the CAD, computer-aided design, models of a category, such as sedan or minivan), with about 1,200 vertices.

Why approximate? Efficiency: rendering a coarse mesh of features is much cheaper than rendering photorealistic pixels, and the comparison needs no model of texture or lighting, so inference is efficient.

The approximations: features, factorization, weak geometry



a cuboid mesh:
vertices on a grid
over the faces of a box

Figure 17: A cuboid mesh: vertices (purple dots) on a regular grid over the six faces of a box (blue lines), drawn for these notes with 104 vertices; the meshes of the paper have about 1,200.

Contrastive learning trains a joint feature extractor

The feature extractor and the vertex features are learned from training images in which the 3D pose of the object is annotated, such as those of PASCAL3D+ (Xiang et al., 2014), a dataset of photographs of 12 object categories, each annotated with the object's 3D pose. With the pose known, each mesh vertex can be projected into each training image, so for every vertex we know which position of each image shows it. These are correspondences across images: the same point of the object, seen in different buses under different lighting and backgrounds.

One feature extractor is shared by all the training images of a category (a joint feature extractor). It is trained by contrastive learning: its features are pulled together for corresponding points and pushed apart for different points. The double arrows in the figure stand for this comparison between the feature maps of different images.

Contrastive learning trains a joint feature extractor

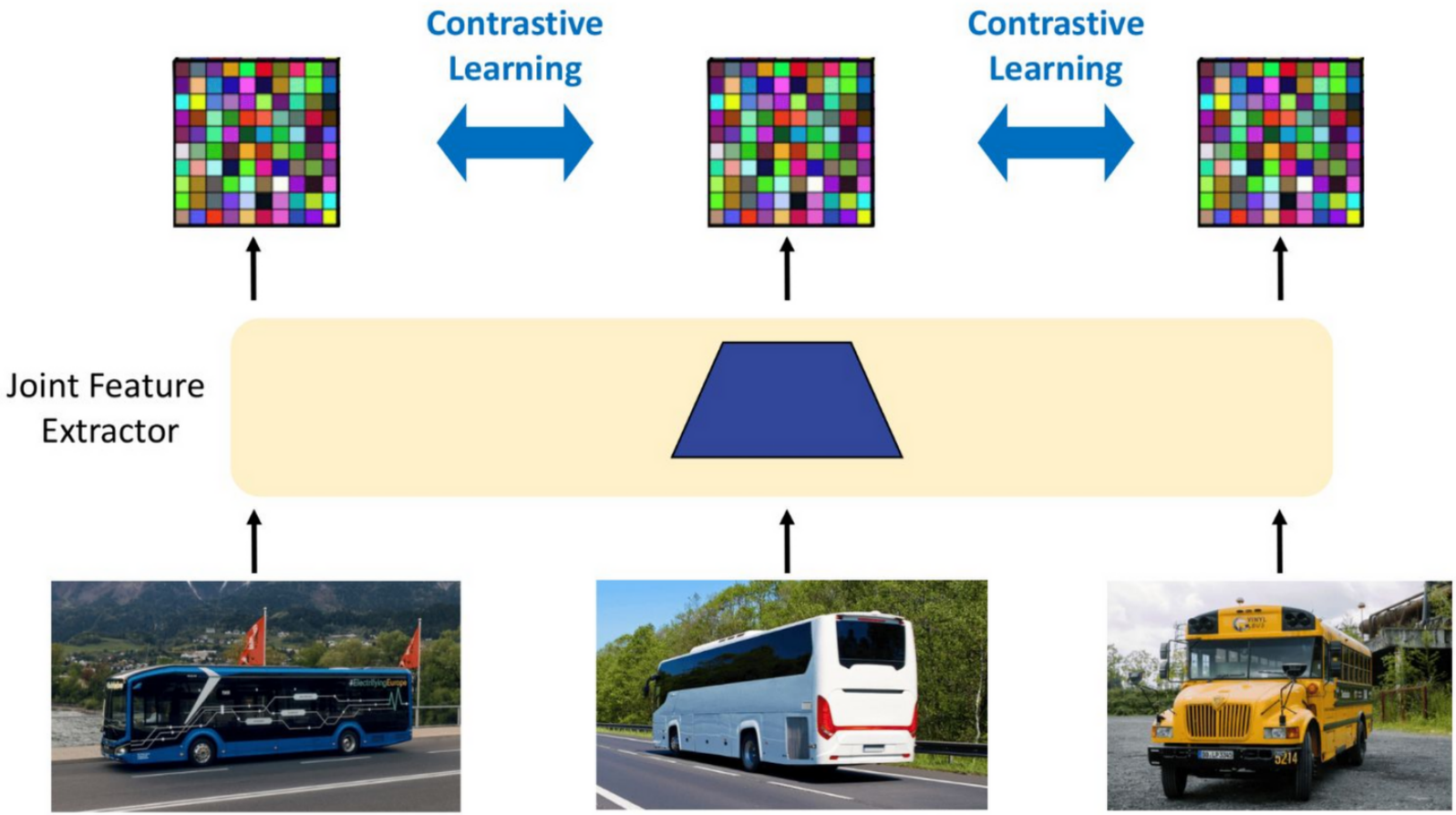


Figure 18: Contrastive learning (A p. 23). Three images of buses pass through one joint feature extractor (the trapezoid); their feature maps (colored grids) are compared with each other (double arrows).

Contrastive learning follows three rules for the features

Contrastive learning of the feature extractor follows three rules.

1. Features of the same point should be similar: the front corner of the bus should give the same feature in every image, whatever the bus's color and the lighting.
2. Features of different points should be dissimilar: two different points of one bus should give different features, so that each feature identifies its vertex.
3. Features on the object should be different from the background: otherwise the background could be mistaken for the object.

Rule 1 makes the features invariant to nuisances, factors such as color, lighting and background that change the image but not the object; rule 2 makes them discriminative between vertices, and rule 3 lets the background model explain clutter and occluders.

Contrastive learning follows three rules for the features

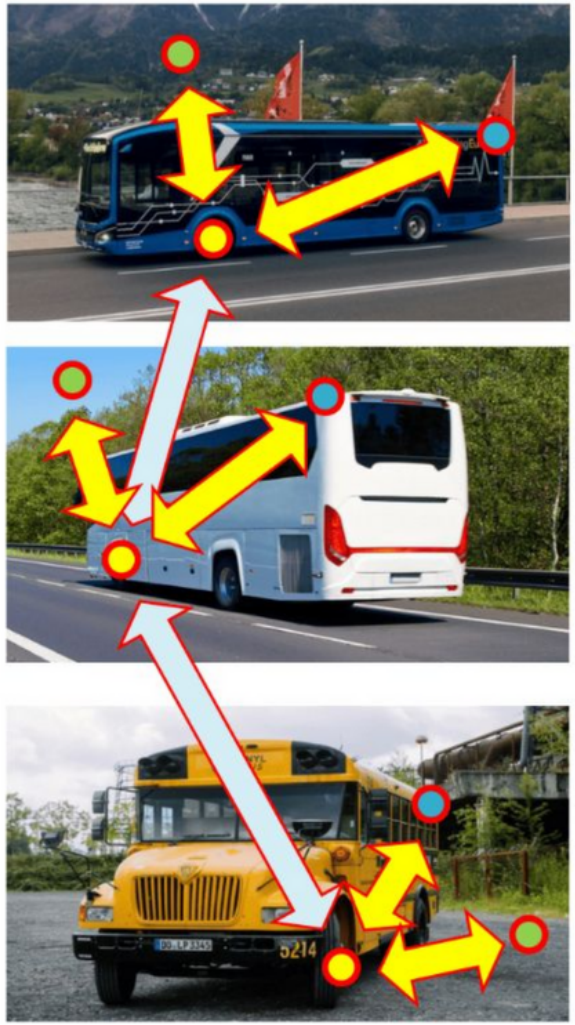


Figure 19: The three rules on three buses (A p. 26). Circles mark points; the light arrows join the same point in different images (rule 1); the yellow arrows join different points of one bus, or a point of the bus and the background (rules 2 and 3).

The contrastive losses push features apart

The general idea of the contrastive loss is to train the feature extractor so that the individual feature vectors on the object are distinct from each other, as well as from the background. NeMo uses two losses (Wang et al., 2021):

$$\mathcal{L}_{Feature}(F, \mathcal{FG}) = -\sum_{i \in \mathcal{FG}} \sum_{i' \in \mathcal{FG} \setminus \{i\}} \|f_i - f_{i'}\|^2,$$

$$\mathcal{L}_{Back}(F, \mathcal{FG}, \mathcal{BG}) = -\sum_{i \in \mathcal{FG}} \sum_{j \in \mathcal{BG}} \|f_i - f_j\|^2.$$

Here i and i' are two foreground positions, showing two different vertices, and j is a background position. (In the likelihood, i' was a background position; here the paper uses j for those.) The losses only make sense for features of bounded length; in the official code the features are compared through dot products divided by a temperature of 0.07, with unit-length features. Minimizing $\mathcal{L}_{Feature}$ makes the squared distances between the features of different vertices large, which is rule 2; minimizing $\mathcal{L}_{Back}$ makes object features far from background features, which is rule 3.

The contrastive losses push features apart

The total loss adds the maximum-likelihood term of the generative model:

$$\mathcal{L} = \mathcal{L}_{ML} + \mathcal{L}_{Feature} + \mathcal{L}_{Back}.$$

$\mathcal{L}_{ML}$, the negative log-likelihood, pulls each foreground feature toward the feature θ_r of its vertex, which is rule 1: all the images that show vertex r are pulled toward the same vector. The three terms are added without weights.

The vertex features θ_r are learned together with the extractor. In the official code they are kept in a memory bank and updated from the image features of their vertex in each batch, with a momentum of 0.9, so that they change slowly while the extractor is trained.

With unit features the loss is a softmax over vertex features

Later neural mesh models, NOVUM (Jesslen et al., 2024) and 3D-CompNets (Yuan et al., 2025), normalize every feature to unit length and replace the Gaussian by a von Mises-Fisher distribution, the analogue of a Gaussian on the sphere:

$$P(f_k \mid C_k) = c(\kappa)\, e^{\kappa\, f_k \cdot C_k}, \qquad \|f_k\| = \|C_k\| = 1.$$

f_k is the image feature at the position where vertex k projects and C_k is the feature of vertex k, the θ_r of NeMo. The concentration κ controls how sharply the probability falls as f_k turns away from C_k, and $c(\kappa)$ normalizes it. Since both vectors have unit length, $f_k \cdot C_k$ is the cosine of the angle between them, so the likelihood depends only on that angle.

With unit features the loss is a softmax over vertex features

The loss of vertex k in an image is then (Yuan et al., 2025)

$$\mathcal{L}(k) = -\log \frac{P(f_k \mid C_k)}{\sum_{C_l \in \mathcal{C}} P(f_k \mid C_l) + \sum_{\beta_n \in \mathcal{B}} P(f_k \mid \beta_n)},$$

where $\mathcal{C}$ is the set of all the vertex features of the category and $\mathcal{B}$ a set of background features β_n sampled from the background. If the background features are scored by the same von Mises-Fisher distribution, with the same κ and unit β_n, $c(\kappa)$ cancels and the ratio is a softmax of the scores $\kappa f_k \cdot C_l$ and $\kappa f_k \cdot \beta_n$: the loss treats the feature f_k as an input to classify among all vertices and the background, with the right answer k. Minimizing it makes f_k match its own vertex feature better than every other vertex (rule 2) and the background (rule 3), and moves it toward C_k (rule 1). The paper adds a second term of the same form that contrasts f_k with the vertex features of other categories.

Render-and-compare finds the pose and the occluded parts

To estimate the pose of an object in a test image, NeMo renders and compares (Wang et al., 2021). The CNN (the backbone) computes the feature map F of the image, and the neural mesh model is rendered at the current camera parameters m (azimuth, elevation and in-plane rotation) to the map $\bar{F}$ of predicted features. At each position, two scores are computed: the foreground score, how well the feature fits the vertex feature rendered there, and the background score, how well it fits the clutter feature β.

An occluder covers part of the object, so the image features there fit the background better than the mesh. NeMo therefore lets each foreground position i choose its explanation with a binary variable z_i, whose prior $p(z_i = 1)$ is the probability that a position is visible. The joint probability of the features and of z is

$$p(F, z \mid \mathfrak{N}_y, m, B) = \prod_{i \in \mathcal{FG}} \Big(\big[p(f_i \mid \mathfrak{N}_y, m)\, p(z_i = 1) \big]^{z_i} \times \big[p(f_i \mid B)\, p(z_i = 0) \big]^{1 - z_i} \Big) \prod_{i' \in \mathcal{BG}} p(f_{i'} \mid B).$$

Render-and-compare finds the pose and the occluded parts

Where the background score wins, $z_i = 0$ and the position is predicted occluded; the map of the z_i is the occlusion prediction Z. Occluded positions then contribute only through the background model, so an occluder has little influence on the estimated pose.

The loss, the negative logarithm of this likelihood, is backpropagated (its gradient is computed through the renderer) to the camera pose m, which is optimized by gradient descent; for each pose the z_i take their best values. Because gradient descent finds only a local minimum, the search starts from the best of 144 poses sampled uniformly, 12 azimuths times 4 elevations times 3 in-plane rotations; the object's position and scale in the image are assumed given.

Render-and-compare finds the pose and the occluded parts

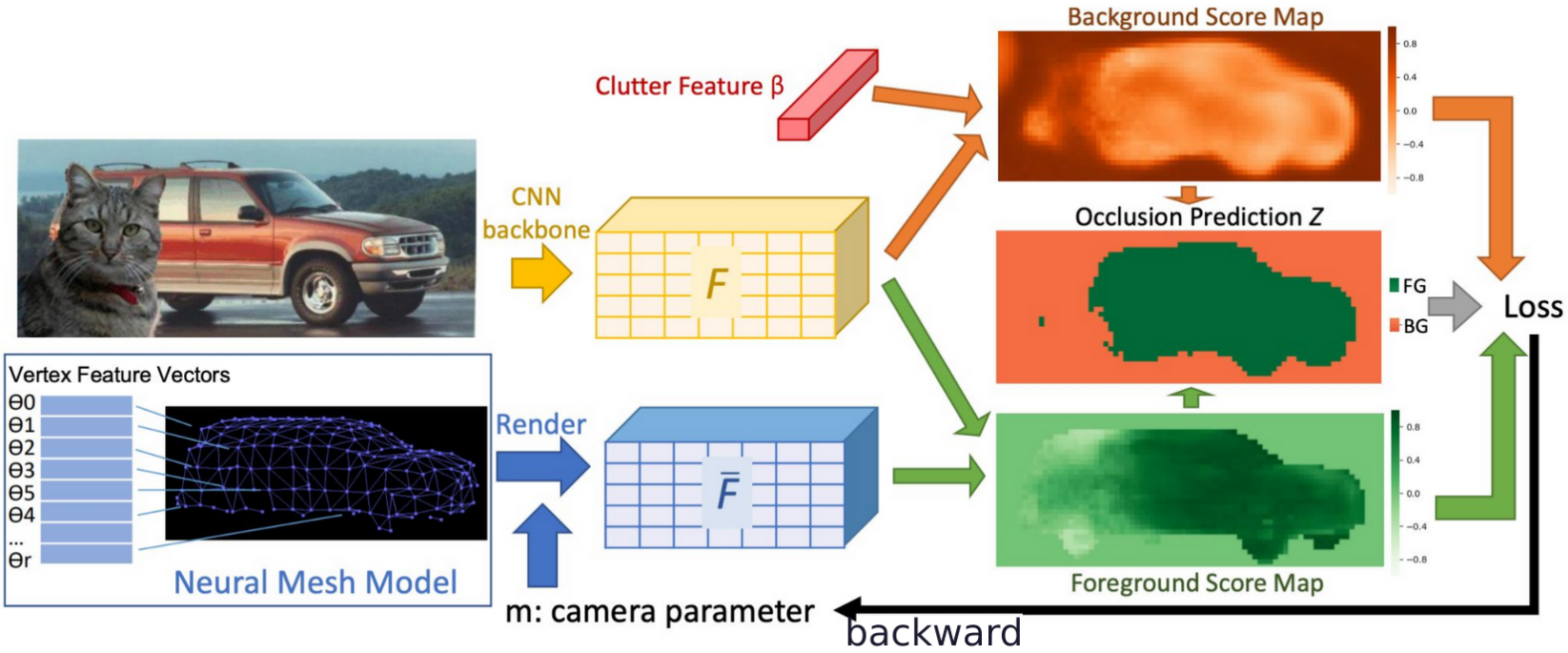


Figure 20: Inference by render-and-compare (A p. 41; Wang et al., 2021). The CNN backbone gives F; the neural mesh model is rendered to $\bar{F}$ at the camera parameter m. The background score map (orange, from the clutter feature β) and the foreground score map (green) give the occlusion prediction Z (green: object, orange: background); the loss is backpropagated to m. The color bars give the scales of the scores.

NeMo-MultiCuboid is the most accurate under occlusion

The table compares NeMo with methods that predict the pose directly, on PASCAL3D+ and on its occluded version. In the occluded version, L0 are the original images, and occluders cover 20–40 % (L1), 40–60 % (L2) and 60–80 % (L3) of the object area.

Pose accuracy is measured three ways. $\text{ACC}_{\pi/6}$ and $\text{ACC}_{\pi/18}$ are the percentages of images whose pose error, the angle between the estimated and the true rotation, is below $\pi/6$ (30 degrees) and $\pi/18$ (10 degrees); MedErr is the median pose error in degrees. Res50-General is a single ResNet-50 that classifies the pose of all categories; Res50-Specific trains one per category; StarMap (Zhou et al., 2018) predicts keypoints and computes the pose from them. NeMo uses a down-sampled CAD mesh of each subtype, NeMo-MultiCuboid one cuboid per subtype and NeMo-SingleCuboid one cuboid per category.

NeMo-MultiCuboid is the most accurate under occlusion

	$ACC_{\pi/6} \uparrow$				$ACC_{\pi/18} \uparrow$			
Occlusion level	L0	L1	L2	L3	L0	L1	L2	L3
Res50-General	88.1	70.4	52.8	37.8	44.6	25.3	14.5	6.7
Res50-Specific	87.6	73.2	58.4	43.1	43.9	28.1	18.6	9.9
StarMap	**89.4**	71.1	47.2	22.9	59.5	34.4	13.9	3.7
NeMo	84.1	73.1	59.9	41.3	60.4	45.1	30.2	14.5
NeMo-MultiCuboid	86.7	**77.2**	**65.2**	**47.1**	**63.2**	**49.9**	**34.5**	**17.8**
NeMo-SingleCuboid	86.1	76.0	63.9	46.8	61.0	46.3	32.0	17.1

MedErr (degrees) $\downarrow$	L0	L1	L2	L3
Res50-General	11.7	17.9	30.4	46.4
Res50-Specific	11.8	17.3	26.1	44.0
StarMap	9.0	17.6	34.1	63.0
NeMo	9.3	15.6	24.1	41.8
NeMo-MultiCuboid	**8.2**	**13.0**	**20.2**	**36.1**
NeMo-SingleCuboid	8.8	13.6	20.9	36.5

Pose estimation on PASCAL3D+ (L0) and occluded PASCAL3D+ (L1–L3), Wang et al. (2021), Table 1; bold: the best in each column.

NeMo-MultiCuboid is the most accurate under occlusion

The table shows two things. On the original images (L0), StarMap is the most accurate at $\pi/6$; the three NeMo variants beat it at the stricter $\pi/18$, and the two cuboid variants have the lowest median error (8.2 and 8.8 degrees, against 9.0 for StarMap). As the occlusion grows, every method gets worse. At $\pi/6$ the NeMo variants lose the fewest points from L0 to L3 (39 to 43, against 44 to 67 for the others): at L3, where most of the object is hidden, NeMo-MultiCuboid is still correct within 30 degrees on 47.1 % of the images, against 22.9 % for StarMap, whose keypoints are hidden by the occluders.

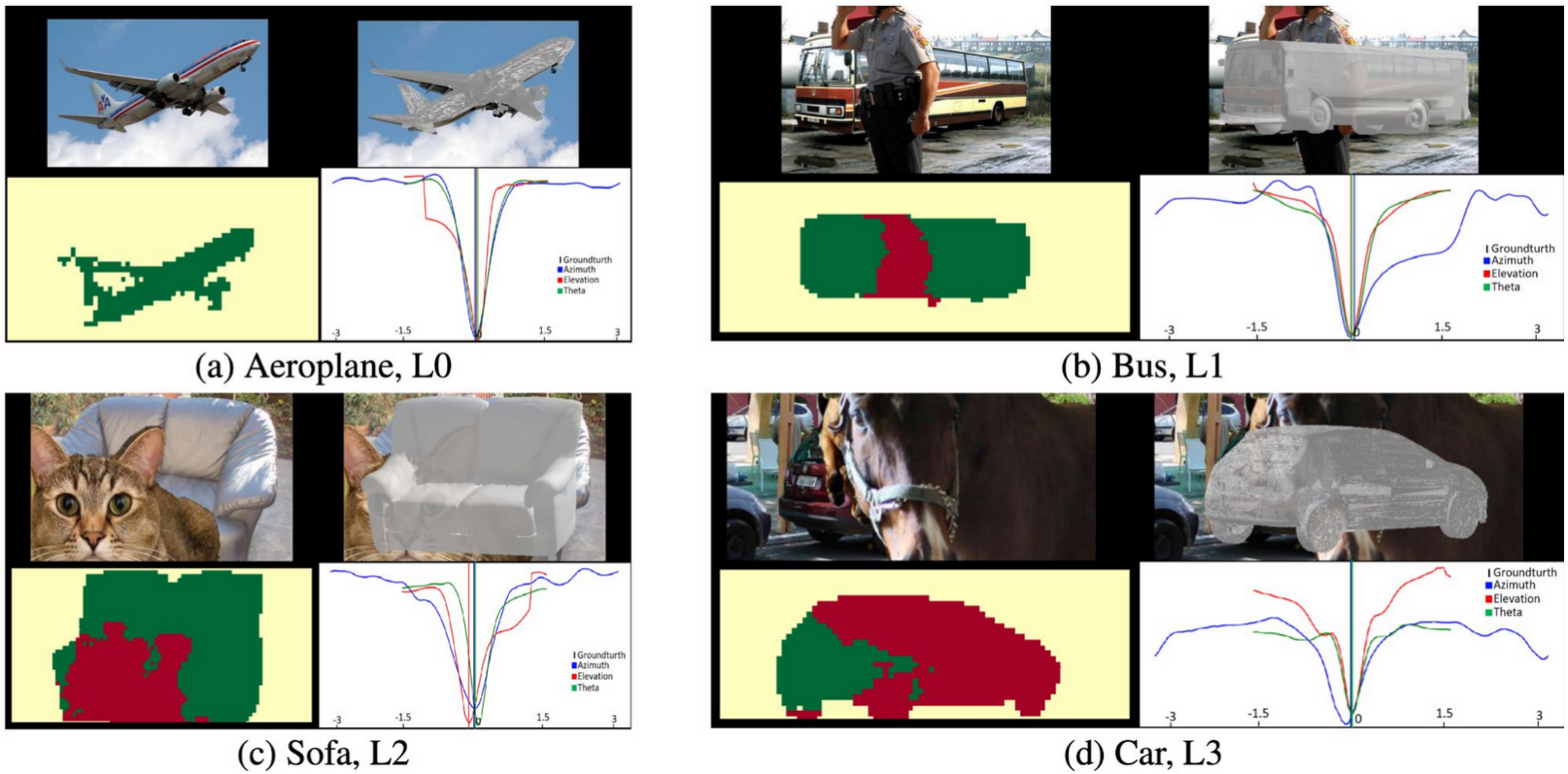
In this table the two cuboid meshes are more accurate than the detailed CAD meshes in every column: for pose, the exact shape matters less than the features, which is what the weak geometry of the approximations assumes.

NeMo finds the pose and marks the occluded parts

The figure shows four examples, one per occlusion level: an aeroplane (L0), a bus (L1), a sofa (L2) and a car (L3). Each example has four panels. The top two are the input image and the image with the mesh rendered at the estimated pose. The bottom two are the occlusion map and a plot of the loss. In the occlusion map, yellow is background, green the visible part of the object, and red the part of the object predicted to be occluded. In the bus, the man standing in front of it is marked red; in the sofa, the cat.

The plot shows the loss as each camera parameter varies alone around the estimate: azimuth in blue, elevation in red and in-plane rotation in green ("Theta" in the legend, unrelated to the vertex features θ_r). The horizontal axis is the offset from the ground truth, from -3 to 3 radians, so the ground truth is at its center; the colored vertical lines mark the final prediction. A sharp, single minimum at the center means that the pose is well determined. The occlusion map covers only the object's outline: occluders outside the object, such as the man's head and shoulders above the bus, are background.

NeMo finds the pose and marks the occluded parts



(a) Aeroplane, L0 (b) Bus, L1

(c) Sofa, L2 (d) Car, L3

Figure 21: Pose estimation with occlusion (A p. 31; Wang et al., 2021, Figure 3): input, rendering at the estimated pose, occlusion map (yellow: background, green: visible object, red: predicted occluded) and the loss as each camera parameter varies, for (a) an aeroplane, L0, (b) a bus, L1, (c) a sofa, L2 and (d) a car, L3.

Out-of-distribution test data change one nuisance at a time

Standard tests use i.i.d. data: the test images are independent and identically distributed, drawn from the same distribution as the training images. Robustness needs out-of-distribution (OOD) test data, which differ from the training data in a controlled way.

OOD-CV (Zhao et al., 2022) collects such images for 10 object categories. Each test image differs from the training data in one nuisance, a factor that changes the image but not the object's class: its shape (a car shaped like a sausage), its pose (an overturned car), its texture (a car painted with lettering), its context (a car under water) or the weather (a car in heavy rain). OOD-CV-v2 (Zhao et al., 2023) adds occlusion as a nuisance. Since only one nuisance changes, a drop in accuracy can be attributed to it.

Out-of-distribution test data change one nuisance at a time

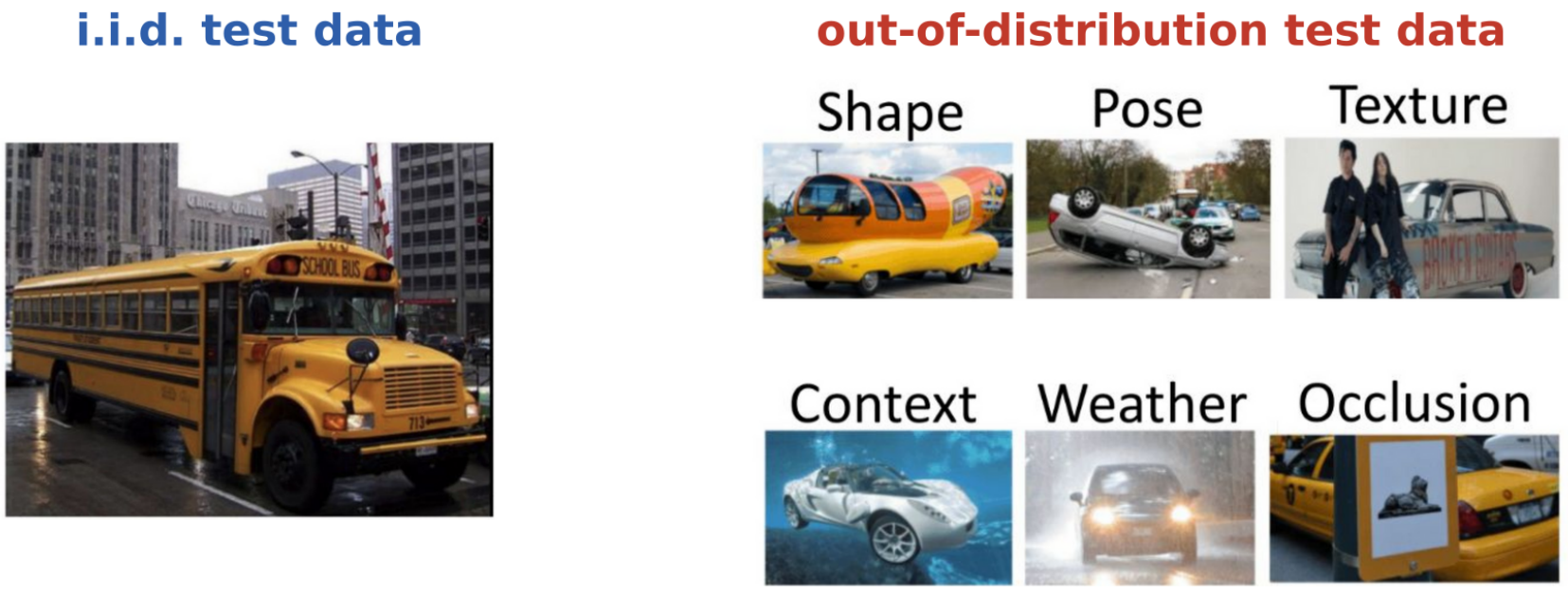


Figure 22: An i.i.d. test image (a school bus in a street) and out-of-distribution test images, one per nuisance: shape, pose, texture, context, weather and occlusion (A p. 32).

Pose accuracy: NeMo leads, by a small margin on OOD-CV

The table compares the pose accuracy of NeMo with that of standard networks trained to predict the pose: a ResNet-50, a Swin-T (shifted-window transformer), a ConvNeXt (a modern CNN) and a ViT-B/16 (vision transformer). A prediction is counted correct if its angular error is below $\pi/18$. The test sets are PASCAL3D+ (P3D+, 12 categories), its occluded version, a corrupted version (the same images with synthetic image corruptions) and OOD-CV.

Among the methods shown, NeMo is the most accurate on every test set. Its lead is large on PASCAL3D+ and on the occluded and corrupted versions, and small on OOD-CV: 21.9 against 21.5 for ViT-B/16. The table is from a manuscript of Jesslen et al., reproduced in the appendix of OOD-CV-v2 (Zhao et al., 2023). The full table also has two rows for the manuscript's own render-and-compare classifier, RCNet: 61.6, 27.2, 43.8, 25.5 and 65.1, 28.8, 43.9, 24.8 on the four test sets, both higher than NeMo on OOD-CV.

Pose accuracy: NeMo leads, by a small margin on OOD-CV

Method	P3D+	occluded P3D+	corrupted P3D+	OOD-CV
ResNet-50	39.0	15.8	15.8	18.0
Swin-T	46.2	16.6	15.6	19.8
ConvNeXt	38.9	14.1	24.1	19.9
ViT-B/16	38.0	15.0	21.3	21.5
NeMo	**62.9**	**30.1**	**43.4**	**21.9**

Pose accuracy (%), angular error below $\pi/18$ (A p. 33; Zhao et al., 2023, appendix B, Table 3). Bold: the best of these rows in each column.

PASCAL3D+ pairs its photographs with CAD models aligned to the objects; the pose annotation is defined by the vertices of these models, as in the figure.

Pose accuracy: NeMo leads, by a small margin on OOD-CV

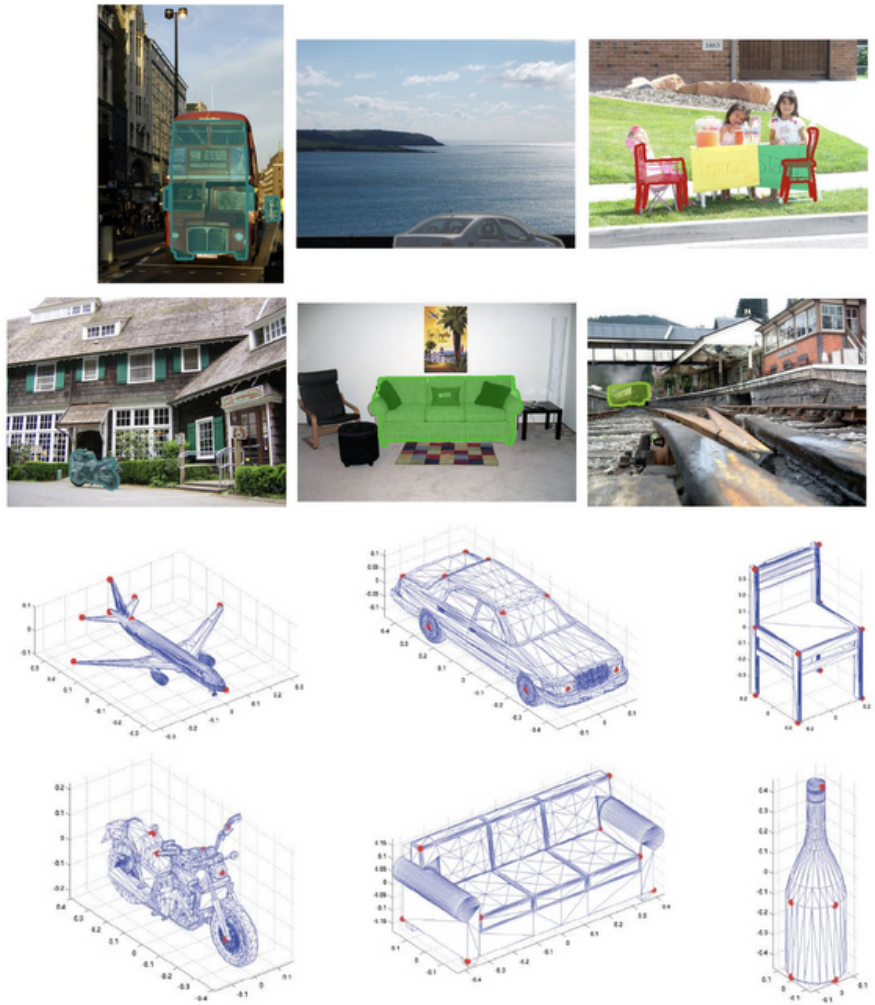


Figure 23: PASCAL3D+ (A p. 33): six of its photographs and six of its CAD models (aeroplane, car, chair, motorbike, sofa, bottle), with the vertices that define the pose annotation.

One render-and-compare model per class gives the class

Render-and-compare can also classify (Jesslen et al., 2023). A shared feature extractor computes one feature map, and every class y has its own 3D object model $\mathfrak{N}_y$, with vertices and a neural texture. Render-and-compare is run with every class model: each finds its best pose and the likelihood of the features at that pose. The class whose model explains the features best is the estimated class $\hat{y}$.

For this, the features and neural textures need to be trained in a discriminative manner: the right class's model must score highest. If the vertex features of a bus and a train were similar, both models would explain a bus image equally well. Jesslen et al. (2023) train them so, as the title of their paper, discriminative render-and-compare, says; the features of different classes are pushed apart, as the features of different vertices are pushed apart within one class.

One render-and-compare model per class gives the class

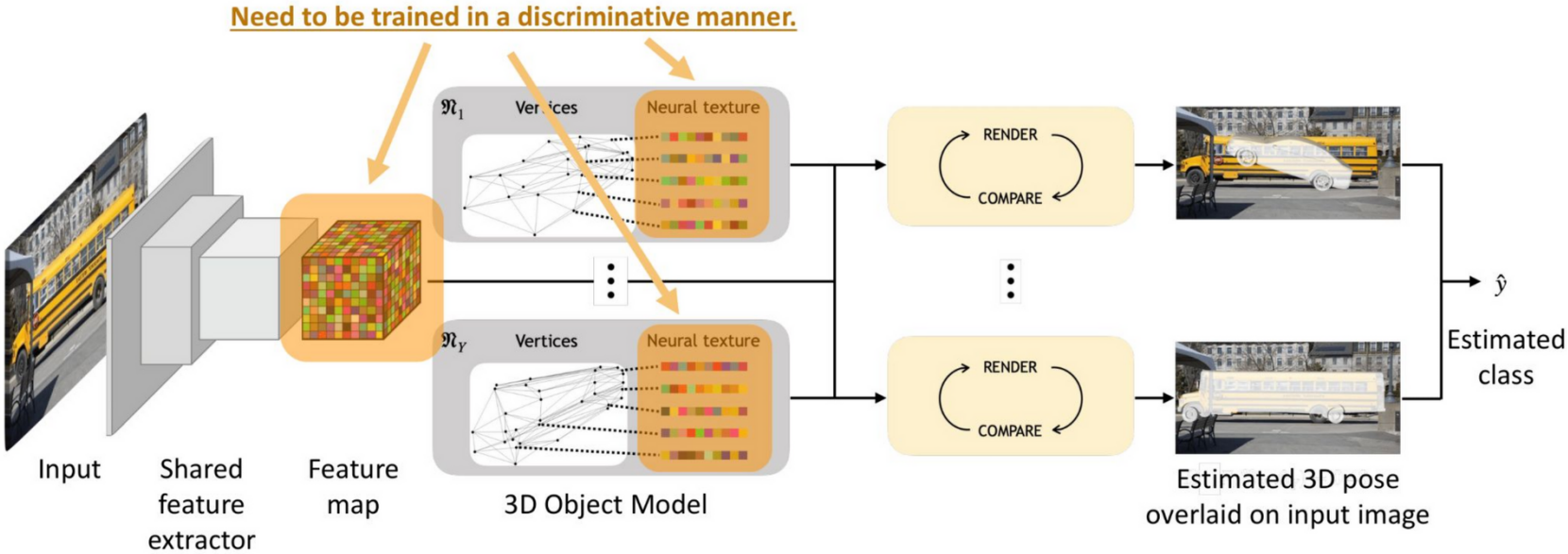


Figure 24: Classification by render-and-compare (A p. 30; Jesslen et al., 2023, Figure 2): input, shared feature extractor, feature map, one 3D object model per class ($\mathfrak{N}_1, \ldots, \mathfrak{N}_Y$), render-and-compare for each (Y is the number of classes), estimated 3D pose and estimated class $\hat{y}$. The orange arrows: the parts that need discriminative training.

NOVUM classifies by matching features, then finds the pose

Rendering every class model at every pose is slow when there are many classes. NOVUM (Jesslen et al., 2024) separates classification from pose estimation. Each class has a neural object volume: a cuboid whose points carry feature vectors, as in the figure (bus, train, bike). The extractor Φ maps the input I to the feature map F.

Classification ignores the geometry: for each class, the score S counts how well the image features match the features of that class, whatever their positions, like a bag of words in text classification, which counts the words of a document regardless of their order. In the example, $S_{\text{Bus}} = 0.88$ and $S_{\text{Train}} = 0.29$, and the bus wins. Then render-and-compare finds the pose α of the chosen class only (α is NOVUM's name for the pose m of NeMo).

NOVUM classifies by matching features, then finds the pose

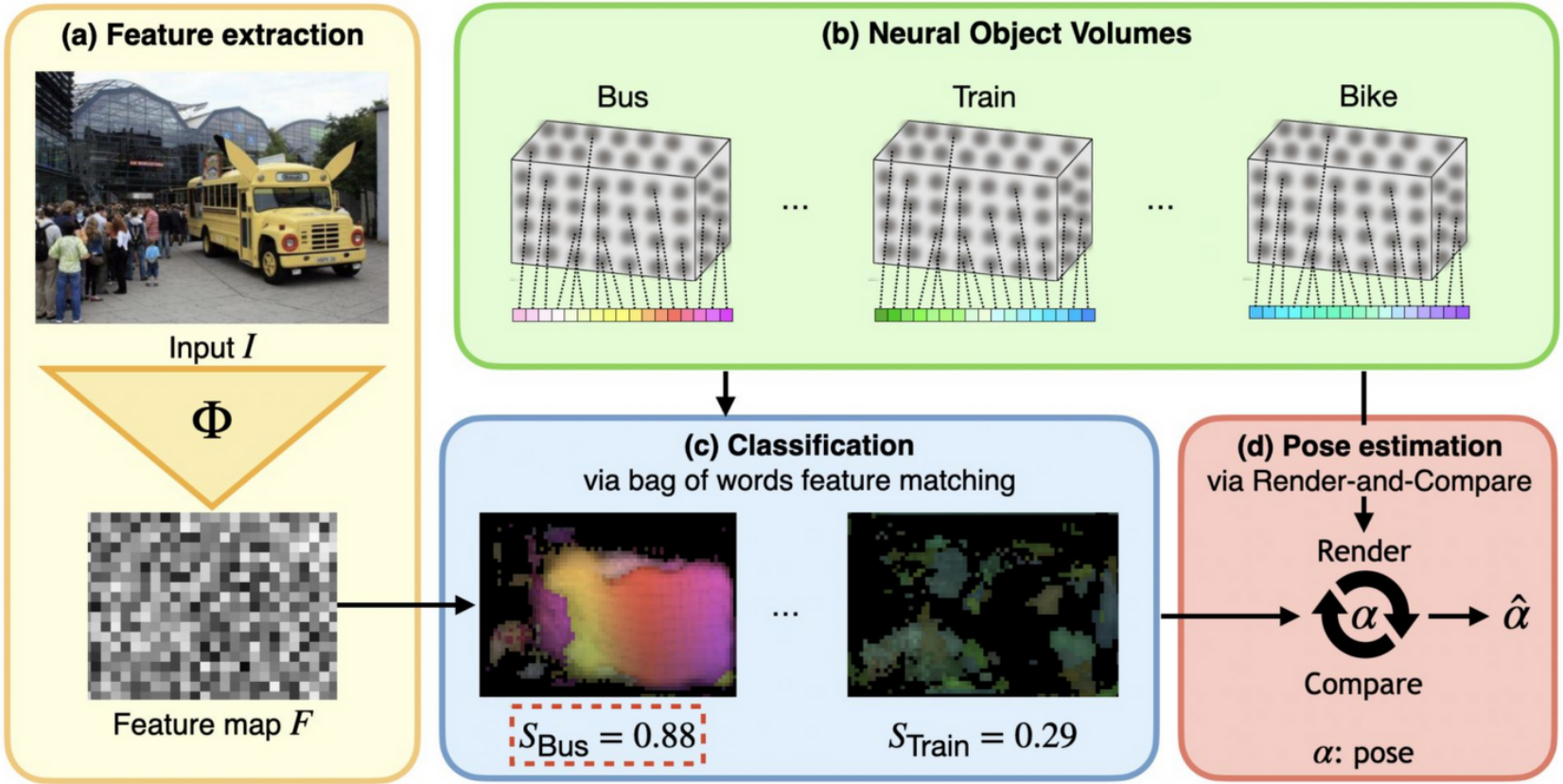


Figure 25: NOVUM (A p. 44; Jesslen et al., 2024): (a) feature extraction Φ of the input I into the feature map F; (b) the neural object volumes of the classes; (c) classification by matching features (the colored maps show where the bus features match; the train's match poorly); (d) pose estimation of the chosen class by render-and-compare.

The learned features form one cluster per class

If classification by feature matching works, the features of different classes must occupy different regions of the feature space. A t-SNE plot (t-distributed stochastic neighbor embedding) shows this: it places each high-dimensional feature as a point in the plane so that features that are close stay close.

In the plot, each point is one learned feature C of the neural object volume of one of the 12 PASCAL3D+ classes, colored by class; the legend of the original is too small to reproduce, so the colors are not named. The features form one cluster per class, with little overlap.

The learned features form one cluster per class



Figure 26: t-SNE of the learned features of the 12 PASCAL3D+ classes, one color per class (A p. 45; Jesslen et al., 2024).

3D-CompNets scale neural mesh models to 188 categories

3D-CompNets (3D compositional networks; Yuan et al., 2025) scale neural mesh models to 188 categories of ImageNet3D (Ma et al., 2024), a dataset of 200 categories with 3D pose annotations. They use von Mises-Fisher likelihoods, cuboid meshes and render-and-compare, with a ViT-B/14 backbone (a base-size vision transformer that cuts the image into patches of 14×14 pixels).

They are compared with DINOv2, a self-supervised vision transformer with the same backbone. Trained on 178 categories and tested on 10 categories held out from training, 3D-CompNets estimate the pose better than DINOv2 (45.5 % against 38.9 % within $\pi/6$; a ResNet-50 reaches 15.0 %). On in-distribution images the two are about equally accurate in classification (93.5 % and 92.4 %) and pose (57.6 % and 56.9 %). Under occlusion 3D-CompNets are slightly more accurate (43.7 % against 41.7 % in classification, 29.5 % against 28.9 % in pose), and under image corruptions clearly more accurate in classification (59.2 % against 52.4 %). The tasks are object-centric: the object's location can be given by Grounding DINO (Liu et al., 2024), an open-vocabulary detector that finds the objects named in a text prompt, or another detection model.

3D-CompNets scale neural mesh models to 188 categories

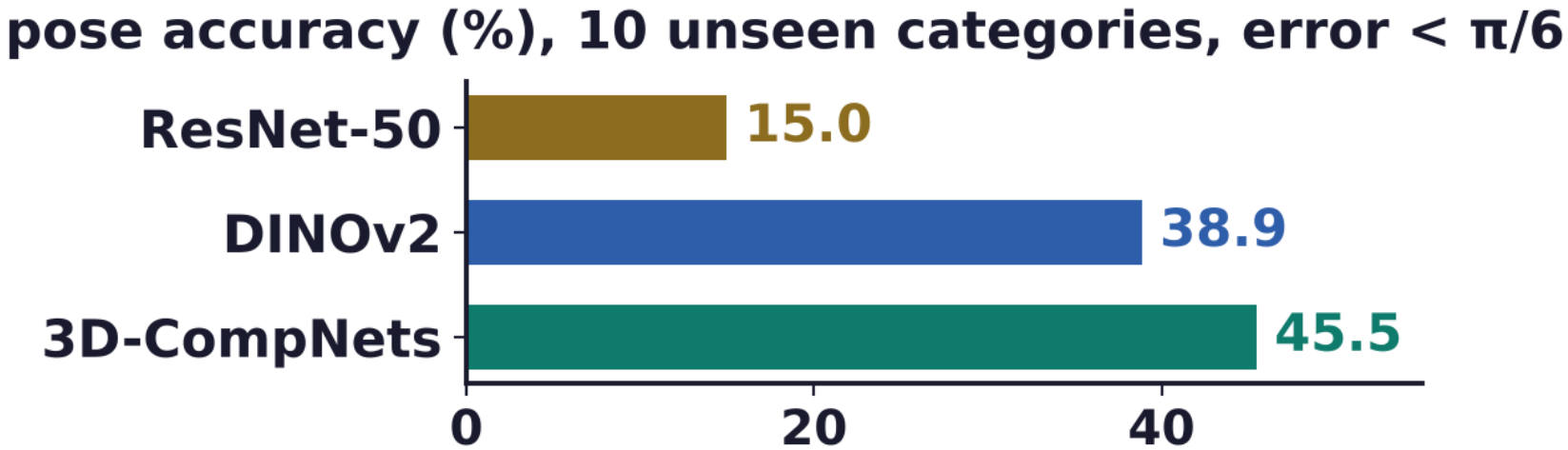


Figure 27: Pose accuracy within $\pi/6$ on 10 categories unseen in training, averaged over the categories (Yuan et al., 2025, Table 3). DINOv2 and 3D-CompNets use a ViT-B/14 backbone.

Summary

- Analysis by synthesis inverts the synthesis of images: $P(W \mid I) \propto P(I \mid W)\, P(W)$.
- Low-level cues make bottom-up proposals; high-level generative models validate them top-down.
- It offers the potential to perform multiple visual tasks in a consistent way.
- Discriminative networks lack robustness to 3D changes and to changes of image components.
- It needs object models conditioned on the 3D world state, hard to learn and hard to invert.
- Neural mesh models approximate it: vertex features, a factorized likelihood, a cuboid mesh.
- Contrastive learning makes the vertex features distinct from each other and from the background.
- Render-and-compare estimates 3D pose robustly under occlusion and extends to classification.

References

[1] A. L. Yuille. Approximate analysis by synthesis. Course slides, Probabilistic Models of the Visual Cortex, Johns Hopkins University, 2025 (A).

[2] U. Grenander. *Lectures in Pattern Theory, Vol. 2: Pattern Analysis*. Springer, New York, 1978.

[3] D. Mumford. Pattern theory: a unifying perspective. In *First European Congress of Mathematics, Vol. I*, pages 187–224. Birkhäuser, 1994.

[4] A. Yuille and D. Kersten. Vision as Bayesian inference: analysis by synthesis? *Trends in Cognitive Sciences*, 10(7):301–308, 2006.

[5] P. Sinha and E. Adelson. Recovering reflectance and illumination in a world of painted polyhedra. *ICCV*, pages 156–163, 1993.

[6] W. Qiu and A. Yuille. UnrealCV: connecting computer vision to Unreal Engine. *ECCV Workshops*, LNCS 9915, pages 909–916, 2016.

References

[7] M. A. Alcorn, Q. Li, Z. Gong, C. Wang, L. Mai, W.-S. Ku and A. Nguyen. Strike (with) a pose: neural networks are easily fooled by strange poses of familiar objects. *CVPR*, pages 4845–4854, 2019.

[8] A. Rosenfeld, R. Zemel and J. K. Tsotsos. The elephant in the room. arXiv:1808.03305, 2018.

[9] R. Geirhos, P. Rubisch, C. Michaelis, M. Bethge, F. A. Wichmann and W. Brendel. ImageNet-trained CNNs are biased towards texture; increasing shape bias improves accuracy and robustness. *ICLR*, 2019.

[10] C. Michaelis, B. Mitzkus, R. Geirhos, E. Rusak, O. Bringmann, A. S. Ecker, M. Bethge and W. Brendel. Benchmarking robustness in object detection: autonomous driving when winter is coming. arXiv:1907.07484, 2019.

[11] A. Kortylewski, J. He, Q. Liu and A. L. Yuille. Compositional convolutional neural networks: a deep architecture with innate robustness to partial occlusion. *CVPR*, pages 8940–8949, 2020.

References

[12] P. Paysan, R. Knothe, B. Amberg, S. Romdhani and T. Vetter. A 3D face model for pose and illumination invariant face recognition. *AVSS*, pages 296–301, 2009.

[13] B. Egger, S. Schönborn, A. Schneider, A. Kortylewski, A. Morel-Forster, C. Blumer and T. Vetter. Occlusion-aware 3D morphable models and an illumination prior for face image analysis. *IJCV*, 126(12):1269–1287, 2018.

[14] A. Wang, A. Kortylewski and A. Yuille. NeMo: neural mesh models of contrastive features for robust 3D pose estimation. *ICLR*, 2021.

[15] Y. Xiang, R. Mottaghi and S. Savarese. Beyond PASCAL: a benchmark for 3D object detection in the wild. *WACV*, pages 75–82, 2014.

[16] X. Zhou, A. Karpur, L. Luo and Q. Huang. StarMap for category-agnostic keypoint and viewpoint estimation. *ECCV*, pages 318–334, 2018.

[17] B. Zhao, S. Yu, W. Ma, M. Yu, S. Mei, A. Wang, J. He, A. Yuille and A. Kortylewski. OOD-CV: a benchmark for robustness to out-of-distribution shifts of individual nuisances in natural images. *ECCV*, LNCS 13668, pages 163–180, 2022.

References

[18] B. Zhao, J. Wang, W. Ma, A. Jesslen, S. Yang, S. Yu, O. Zendel, C. Theobalt, A. Yuille and A. Kortylewski. OOD-CV-v2: an extended benchmark for robustness to out-of-distribution shifts of individual nuisances in natural images. arXiv:2304.10266, 2023.

[19] A. Jesslen, G. Zhang, A. Wang, A. Yuille and A. Kortylewski. Robust 3D-aware object classification via discriminative render-and-compare. arXiv:2305.14668, 2023.

[20] A. Jesslen, G. Zhang, A. Wang, W. Ma, A. Yuille and A. Kortylewski. NOVUM: neural object volumes for robust object classification. *ECCV*, 2024.

[21] X. Yuan, G. Zhang, P. Kaushik, A. Jesslen, A. Kortylewski and A. Yuille. Scaling 3D compositional models for robust classification and pose estimation. *ICCV*, pages 6406–6415, 2025.

[22] W. Ma, G. Zhang, Q. Liu, G. Zeng, A. Kortylewski, Y. Liu and A. Yuille. ImageNet3D: towards general-purpose object-level 3D understanding. *NeurIPS Datasets and Benchmarks*, 2024.

[23] S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, Q. Jiang, C. Li, J. Yang, H. Su, J. Zhu and L. Zhang. Grounding DINO: marrying DINO with grounded pre-training for open-set object detection. *ECCV*, 2024.